

Respuestas del 1er parcial

Defensa oral del backend · Banco de 50 preguntas

StockFlow — Inventario, compras internas y movimientos de almacén
(PA-06)

Programación Aplicada 2026-2 · Spring Boot 3.5 · Java 21 · PostgreSQL
16

Código analizado	Rama Hector , commit 629886b
Fecha	3 de octubre de 2026
Contenido	50 preguntas con respuesta, ubicación en el código, demostración y repregunta · 5 desafíos prácticos · checklist

Las rutas de archivos cuelgan de [intellij/stockflow-backend/src/main/java/com/stockflow/](#) salvo que se indique otra. Los números de línea corresponden al commit indicado; si el código cambia, busca la clase o el método por su nombre (Ctrl+N y Ctrl+F12 en IntelliJ).

Índice

Cómo usar esta guía	4
----------------------------	----------

Mapa del proyecto y preparación de la demo	5
---	----------

Bloque A — Arquitectura, hexagonal, MVC y Spring	7
---	----------

01 (C) — Dibuja la arquitectura real de tu proyecto y explica el recorrido entre sus...	7
02 — ¿Qué significa realmente 'arquitectura hexagonal' en tu backend y qué probl...	8
03 — Diferencia dominio, aplicación e infraestructura usando clases concretas de...	8
04 — ¿Cuáles son tus puertos de entrada y de salida? Señálalos en el código.	9
05 (C) — Explica la regla de dependencias del monolito hexagonal y comprueba si tu p...	10
06 — ¿MVC y arquitectura hexagonal son lo mismo? ¿Cómo conviven en Spring Boot?	10
07 — En una API REST con Spring MVC, ¿dónde están Model, View y Controller?	11
08 — ¿Qué hace @SpringBootApplication y qué ocurre cuando ejecutas main()?	11
09 — Explica IoC, inyección de dependencias y por qué prefieres inyección por co...	12
10 — Haz una auditoría de todas las anotaciones Spring/JPA de tu proyecto: clasi...	12

Bloque B — HTTP, MVC y flujo CRUD de punta a punta	14
---	-----------

11 (C) — Explica el recorrido exacto de una solicitud desde navegador/Postman/Swagge...	14
12 — ¿Qué diferencia hay entre @Controller y @RestController? ¿Qué aporta @Reque...	15
13 — Diferencia @PathVariable, @RequestParam y @RequestBody con endpoints reales.	15
14 — ¿Swagger, Postman y un navegador 'pasan por caminos distintos' dentro de tu...	16
15 (C) — POST: explica línea por línea qué ocurre al crear una EntidadPadre o Entida...	16
16 (C) — GET por id: explica todo el flujo, incluido el caso en que el registro no e...	17
17 — GET de listado: ¿qué cambia respecto al GET por id y qué riesgos aparecen c...	18
18 (C) — PUT: explica el flujo completo de una actualización y por qué PUT debe ser...	18
19 (C) — DELETE: explica el flujo completo y qué ocurre con los hijos en una relació...	19
20 — Explica los códigos HTTP 200, 201, 204, 400, 404, 409 y 500 en el contexto...	20
21 — ¿Qué hace @Valid y en qué momento se ejecutan @NotNull, @NotBlank, @Size u...	20
22 — ¿Cómo centralizas errores con @RestControllerAdvice y @ExceptionHandler?	21
23 — ¿Para qué sirve @Transactional y dónde debería estar el límite de la transa...	21
24 — ¿Qué es CORS y por qué puede fallar React en el navegador aunque Postman fu...	22

Bloque C — JPA, Hibernate, cardinalidad y PostgreSQL	23
---	-----------

25 — ¿Qué hacen @Entity y @Table y qué significa que una clase sea gestionada po...	23
26 — Explica @Id y @GeneratedValue. ¿Cómo se genera tu identificador en PostgreSQL?	23
27 — ¿Qué puede expresar @Column y qué diferencia hay entre esa configuración y...	24
28 (C) — Representa en tablas la cardinalidad uno-a-muchos de tus dos entidades.	24
29 (C) — Explica @OneToMany y @ManyToOne usando exactamente tu relación.	25

30 (C) — ¿Cuál es el lado propietario de la relación y qué significan mappedBy y @Jo...	26
31 — Explica cascade y cada CascadeType que realmente utilizas. ¿Por qué Cascade...	26
32 — ¿Qué hace orphanRemoval=true y en qué se diferencia de CascadeType.REMOVE?	27
33 (C) — Explica FetchType.LAZY y EAGER, incluyendo los defaults de @OneToMany y @Ma...	27
34 — En una relación bidireccional, ¿por qué conviene mantener sincronizados amb...	28
35 (C) — ¿Por qué una relación bidireccional puede generar recursión infinita al ser...	28
36 — ¿Qué es el persistence context y qué significa dirty checking?	29
37 — Diferencia los estados transient, managed, detached y removed de una entida...	30
38 — ¿Qué es el problema N+1 y cómo podrías detectarlo y resolverlo?	30
39 — Explica spring.jpa.hibernate.ddl-auto y por qué 'update' no es una estrateg...	31
40 — Desde repository.save() hasta PostgreSQL: explica cómo JPA/Hibernate termin...	31

Bloque D — DTO, record, servicios y repositorios 33

41 — ¿Por qué no conviene exponer tus @Entity directamente como contrato de la API?	33
42 — Diferencia Request DTO y Response DTO. ¿Por qué no siempre deben tener los...	33
43 (C) — ¿Qué es un record de Java y por qué es útil para DTOs?	34
44 — ¿Dónde y cómo conviertes Entity ↔ DTO? Defiende tu estrategia de mapeo.	34
45 (C) — ¿Qué responsabilidades pertenecen al Service/caso de uso y cuáles NO?	35
46 — Diferencia un puerto de repositorio del dominio/aplicación y un JpaReposito...	35
47 — Explica repository.findById(), Optional, consultas derivadas y @Query.	36

Bloque E — Validación, pruebas y evolución a microservicios 37

48 — Separa tres niveles de validación: DTO/Bean Validation, reglas de negocio y...	37
49 — ¿Cómo probarías Controller, Service y Repository sin mezclar responsabida...	37
50 (C) — Defiende por qué un monolito hexagonal modular es una base sólida para empe...	38

Los 5 desafíos prácticos 40

Desafío 1 — Trazado en vivo	40
Desafío 2 — Romper una anotación	40
Desafío 3 — Error intencional	40
Desafío 4 — SQL y cardinalidad	41
Desafío 5 — Extracción futura	41

Señales de alerta y checklist final 42

Cómo usar esta guía

- Esta guía responde las **50 preguntas** y los **5 desafíos** del *Banco de 50 preguntas — Defensa oral del backend* (Programación Aplicada 2026-2) usando **el código real de StockFlow** (rama **Hector**).
- Cada pregunta tiene: **Qué responder** (la respuesta, con tus palabras), **Muéstralo en el código** (archivo y línea para abrir en IntelliJ), a veces **Demuéstralo** (algo para ejecutar) y la **repregunta de presión** del docente con su respuesta.
- **No la memorices.** El reglamento dice: "*una definición memorizada sin poder ubicarla y justificarla en el propio código se considera respuesta incompleta*". Úsala así: lee la pregunta, intenta responder en voz alta **sin mirar**, abre el archivo indicado en IntelliJ, y recién entonces compara con la guía.
- Las preguntas marcadas **CRÍTICA** son 16. Fallar dos o más obliga a una repregunta adicional: son las que más conviene dominar.

Las 5 exigencias de cada respuesta

Exigencia	Qué debes hacer
Definir	Explicar el concepto con palabras propias, sin leer.
Ubicar	Mostrar en qué paquete, clase, método o anotación aparece.
Recorrer	Explicar qué pasa antes y después de ese punto en el flujo.
Justificar	Por qué se hizo así y qué alternativa existía.
Predcir	Qué ocurre si se elimina, cambia o falla esa pieza.

Cómo se toma el examen

- Se eligen **10 a 12 preguntas al azar**, incluyendo siempre: 1 de arquitectura, 1 de MVC/HTTP, 1 de POST, 1 de GET/PUT/DELETE, **2 de JPA** y 1 de DTO/servicio/repositorio.
- En 3 o 4 preguntas te pedirán **abrir el IDE** y señalar el código exacto.
- En al menos una tendrás que **dibujar en la pizarra** el flujo completo: cliente → Spring MVC → aplicación → persistencia → PostgreSQL → respuesta.
- No se acepta "Spring lo hace": hay que nombrar el componente (DispatcherServlet, Jackson, Hibernate...) y su responsabilidad.
- Cada pregunta vale 0 a 2 puntos: 85-100 preparado; 70-84 defendible con vacíos; 50-69 insuficiente; menos de 50 no domina su backend.

Fórmula para responder cualquier pregunta

1. Defino con mis palabras (1-2 frases). **2.** Digo dónde está en StockFlow y lo abro. **3.** Recorro qué pasa antes y después. **4.** Justifico por qué así y qué alternativa descarté. **5.** Predigo qué fallaría si lo cambio o lo quito.

Mapa del proyecto

Qué	Dónde
Arranque	<code>StockFlowApplication.java</code>
Módulo padre: categorías	<code>category/</code> (<code>domain</code> , <code>application/service/CategoriaService</code> , <code>infrastructure/adapters/in/web/CategoriaController</code> , <code>out/persistence/...</code>)
Módulo hijo: productos	<code>product/</code> (<code>Producto</code> , <code>ProductoService</code> , <code>ProductoController</code> , <code>ProductoPersistenceAdapter</code> , <code>ProductoJpaEntity</code>)
Ubicaciones	<code>location/</code> (<code>Ubicacion</code> , <code>UbicacionService</code> , <code>UbicacionController</code>)
Inventario (lectura)	<code>inventory/</code> (<code>InventarioConsultaService</code> , <code>InventarioController</code> , <code>SpringDataStockRepository</code> con <code>@Query</code>)
Transversal	<code>shared/</code> (<code>GlobalExceptionHandler</code> , <code>ApiErrorResponse</code> , <code>CorsConfig</code> , <code>OpenApiConfig</code> , excepciones base)
Configuración	<code>src/main/resources/application.yml</code>
Migraciones	<code>src/main/resources/db/migration/V1, V2, V3</code>
Pruebas	<code>src/test/java/com/stockflow/</code> (57 pruebas)
Pruebas HTTP	<code>intellij/stockflow-backend/requests.http</code>
Decisiones	<code>docs/03-decisions/ADR-001 a ADR-008</code>

Flujo que debes poder dibujar

Cliente (Postman · Swagger · React) → Tomcat → DispatcherServlet → HandlerMapping → **ProductoController** (adaptador IN) → Jackson + @Valid → **RegistrarProductoUseCase** (puerto IN) → **ProductoService** (@Transactional) → **ProductoRepositoryPort** (puerto OUT) → **ProductoPersistenceAdapter** (adaptador OUT) → SpringDataProductoRepository → Hibernate → JDBC → **PostgreSQL** → ... y de vuelta: entidad → dominio → DTO → Jackson → 201 + Location

Preparar la demostración (antes de entrar)

- **1. Base de datos:** `systemctl --user start docker-desktop` y luego, en la raíz del proyecto, `docker compose up -d`.
- **2. Backend:** en IntelliJ ejecuta `StockFlowApplication`. En la consola verás Flyway (`Successfully applied ...` o `Schema ... is up to date`) y `Started StockFlowApplication`.
- **3. Probar:** abre `requests.http` (botón ► de cada petición) o `http://localhost:8080/swagger-ui.html`.
- **4. Ver el SQL:** cada petición imprime en la consola los `select`, `insert`, `update` y `delete` (`application.yml`: `org.hibernate.SQL: debug`).
- **5. Ver los datos:** DataGrip → conexión `localhost:5432`, base `stockflow`, usuario `stockflow_admin`; consultas en `datagrip/03_verificacion_y_pruebas.sql`.
- **6. Pruebas:** `mvn test` o clic derecho en `src/test/java` → `Run 'All Tests'`. Con Docker encendido corren las 57.

Códigos HTTP de StockFlow

Código	Cuándo	Ejemplo en StockFlow
200 OK	GET y PUT exitosos	<code>GET /api/productos/1</code>

Código	Cuándo	Ejemplo en StockFlow
201 Created	POST que crea; incluye header Location	POST /api/productos
204 No Content	DELETE exitoso, sin cuerpo	DELETE de una categoría sin productos
400 Bad Request	Petición mal formada (@Valid, JSON, tipo)	codigo vacío; /api/productos/abc; unidad BARRIL
404 Not Found	Recurso o padre inexistente	GET /api/productos/9999; categoryId 9999
409 Conflict	Choque con datos existentes	código PRD-OFI-001 repetido; DELETE de CAT-OFI
422 Unprocessable	Regla de negocio violada	mover producto a categoría inactiva
500 Internal Error	Error no previsto (sin detalles internos)	ProductoControllerTest.errorNoControlado500. ..

Bloque A — Arquitectura, hexagonal, MVC y Spring

Pregunta 01 · CRÍTICA

Dibuja la arquitectura real de tu proyecto y explica el recorrido entre sus paquetes.

Qué responder

- StockFlow es un **monolito modular con arquitectura hexagonal simplificada** (ADR-001). Se despliega como una sola aplicación Spring Boot, pero está dividido en **módulos de negocio**: `category`, `product`, `location`, `inventory` y `shared`.
- Dentro de cada módulo hay tres zonas: `domain` (modelo sin JPA, excepciones y **puertos**), `application/service` (implementa los casos de uso) e `infrastructure/adapters` (`in/web` = controllers y DTO; `out/persistence` = JPA y PostgreSQL).
- Recorrido de un POST de producto: `ProductoController` (adaptador de entrada) → `RegistrarProductoUseCase` (puerto IN) → `ProductoService` (aplicación) → `ProductoRepositoryPort` (puerto OUT) → `ProductoPersistenceAdapter` (adaptador de salida) → `SpringDataProductoRepository` → Hibernate → PostgreSQL.
- **Dirección de las dependencias**: siempre hacia el centro. El controller conoce el puerto IN; el servicio conoce el puerto OUT (una interfaz del dominio); el adaptador de persistencia implementa ese puerto. El dominio no importa nada de Spring MVC ni de JPA.
- No es 'controller-service-repository': el servicio **no** depende de `JpaRepository`, depende de una interfaz propia (`ProductoRepositoryPort`).

Muéstralo en el código

- Árbol de paquetes en IntelliJ: `com.stockflow` → `category`, `product`, `location`, `inventory`, `shared`.
- `product/domain/model/Producto.java` (sin `@Entity`).
- `product/domain/port/in/RegistrarProductoUseCase.java` y `product/domain/port/out/ProductoRepositoryPort.java`.
- `product/application/service/ProductoService.java:24` (`@Service`, implementa los puertos IN).
- `product/infrastructure/adapters/in/web/ProductoController.java` y `.../out/persistence/ProductoPersistenceAdapter.java`.
- Documento de respaldo: `docs/03-decisions/ADR-001-monolito-modular-hexagonal.md`.

Repregunta de presión

Si muevo tu Controller al paquete domain, ¿compila? ¿Qué principio rompes?

Sí compila: Java no impone capas, los paquetes son sólo carpetas. Pero se rompe la **regla de dependencias**: el dominio pasaría a depender de Spring MVC (`@RestController`, `ResponseEntity`, DTO web). El núcleo quedaría acoplado a HTTP y ya no podría probarse ni reutilizarse sin la web. La arquitectura la sostienen las dependencias, no el compilador.

Pregunta 02

¿Qué significa realmente 'arquitectura hexagonal' en tu backend y qué problema intenta resolver?

Qué responder

- Es el estilo **puertos y adaptadores**: el núcleo (dominio + casos de uso) queda en el centro y se comunica con el exterior sólo a través de **interfaces propias** (puertos). Las tecnologías (HTTP, JPA, PostgreSQL) son **adaptadores** intercambiables que implementan o usan esos puertos.
- 'Hexagonal' **no** significa seis capas ni un dibujo literal: el hexágono sólo representa que el núcleo puede tener varios 'lados' de entrada y salida.
- Problema que resuelve: evitar que las reglas de negocio queden mezcladas con detalles técnicos. Así se puede cambiar la base, agregar otra entrada (móvil, mensajería) o **probar el caso de uso sin base de datos**: `ProductoServiceTest` usa un `FakeProductoRepository` que implementa `ProductoRepositoryPort` en memoria.

Muéstralo en el código

- Puerto OUT: `product/domain/port/out/ProductoRepositoryPort.java:8`.
- Adaptador que lo implementa: `product/infrastructure/adapter/out/persistence/ProductoPersistenceAdapter.java`.
- Prueba sin base: `src/test/.../product/application/service/ProductoServiceTest.java` (clase interna `FakeProductoRepository`).

Repregunta de presión

Si PostgreSQL se reemplaza por otra persistencia, ¿qué cambia y qué no?

Cambia sólo `infrastructure/adapter/out/persistence` de cada módulo (entidades JPA, repositorios Spring Data, adaptadores) y la configuración. **No cambian** el dominio (`Producto`, `Categoria`), los puertos ni los servicios. La prueba es que `ProductoService` ya funciona con un repositorio en memoria en los tests.

Pregunta 03

Diferencia dominio, aplicación e infraestructura usando clases concretas de tu proyecto.

Qué responder

- **Dominio** = conceptos y reglas del negocio, sin frameworks: `Producto` (normaliza el código, exige stock mínimo ≥ 0 , precio con 2 decimales), `Categoria`, `Ubicacion`, las excepciones (`CodigoProductoDuplicadoException`) y los puertos.
- **Aplicación** = orquesta un caso de uso: `ProductoService.registrar` consulta la categoría, verifica unicidad y pide guardar, dentro de una transacción.
- **Infraestructura** = detalles técnicos: `ProductoController` (HTTP), `CrearProductoRequest` (JSON), `ProductoJpaEntity` (tabla), `SpringDataProductoRepository` (Spring Data), `GlobalExceptionHandler`.
- Importante: la entidad JPA **no** es el dominio. Tengo dos clases distintas: `Producto` (dominio) y `ProductoJpaEntity` (persistencia), unidas por `ProductoPersistenceMapper`.
- Mezcla reconocida y justificada: `@Service` y `@Transactional` (anotaciones de Spring) están en la capa de aplicación. Es la versión **simplificada** que pide el curso: evita crear configuración manual de beans.

Muéstralo en el código

- Dominio: `product/domain/model/Producto.java:12`.
- Aplicación: `product/application/service/ProductoService.java:39` (`registrar`).
- Infraestructura:
`product/infrastructure/adapter/out/persistence/entity/ProductoJpaEntity.java:23` (`@Entity`).

Repregunta de presión

Si tus @Entity estuvieran en el dominio, ¿qué dependencia introduces? ¿Qué haría una hexagonal más estricta?

El dominio dependería de `jakarta.persistence` y de Hibernate (proxies, constructor vacío, campos mutables). Una hexagonal estricta separa ambos modelos, que es **lo que hace StockFlow**:

`Producto` no tiene anotaciones y `ProductoJpaEntity` vive en infraestructura.

Pregunta 04

¿Cuáles son tus puertos de entrada y de salida? Señálalos en el código.

Qué responder

- **Puertos de entrada** (lo que la aplicación ofrece): `RegistrarProductoUseCase`, `ConsultarProductoUseCase`, `ActualizarProductoUseCase`, `EliminarProductoUseCase`, y los equivalentes de categoría y ubicación, más `ConsultarInventarioUseCase`.
- **Puertos de salida** (lo que la aplicación necesita): `ProductoRepositoryPort`, `CategoriaRepositoryPort`, `UbicacionRepositoryPort`, `InventarioConsultaPort`.
- **Adaptador de entrada**: el controller REST, que depende de los puertos IN (no del servicio concreto). **Adaptador de salida**: `ProductoPersistenceAdapter`, que implementa el puerto OUT usando Spring Data.
- Ventaja: el servicio depende de interfaces propias, con nombres del negocio (`existePorCodigo`, `guardar`), no de detalles externos (`JpaRepository`).

Muéstralo en el código

- `product/domain/port/in/` (4 interfaces) y `product/domain/port/out/ProductoRepositoryPort.java`.
- Controller recibiendo puertos IN por constructor: `ProductoController.java` (campos `registrar`, `consultar`, `actualizar`, `eliminar`).
- Implementación del puerto OUT: `ProductoPersistenceAdapter` implements `ProductoRepositoryPort`.

Repregunta de presión

¿Tu JpaRepository es un puerto de dominio o un detalle de infraestructura?

Es un **detalle de infraestructura**: `SpringDataProductoRepository` extends `JpaRepository` vive en `infrastructure/adapter/out/persistence/repository`. Ningún servicio lo importa; sólo el adaptador. El puerto del dominio es `ProductoRepositoryPort`.

Pregunta 05 · CRÍTICA

Explica la regla de dependencias del monolito hexagonal y comprueba si tu proyecto la respeta.

Qué responder

- Regla: **las capas externas dependen de las internas, nunca al revés**. El dominio no conoce la web ni la persistencia; la aplicación conoce el dominio; la infraestructura conoce ambos.
- Comprobación por imports: `Producto.java` sólo importa excepciones propias y `java.math`. `ProductoService` importa puertos y excepciones (dominio), no `JpaRepository` ni `ResponseEntity`. Ningún controller importa un `JpaRepository`.
- Caso corregido: `CategoriaDemoService` (aplicación) importaba antes un DTO web (`CategoriaDemoResponse`). Ahora devuelve el dominio `Categoria` y el controller hace la conversión con `CategoriaWebMapper.toDemoResponse`.
- **Acoplamiento que sí existe y debo reconocer:** en infraestructura, `product` y `category` se conocen a nivel JPA (`ProductoJpaEntity` → `CategoriaJpaEntity` por `@ManyToOne`; `CategoriaJpaEntity` → `ProductoJpaEntity` por `@OneToMany(mappedBy)`; `ProductoPersistenceAdapter` usa `SpringDataCategoriaRepository.getReferenceById`). Está confinado a infraestructura: ni dominio ni servicios lo ven. Documentado en ADR-001 y ADR-003.

Muéstralo en el código

- Imports de `product/domain/model/Producto.java` (sin Spring, sin JPA).
- Imports de `ProductoService.java`: sólo `domain` de `category` y `product` + `@Service/@Transactional`.
- Corrección de la regla: `category/application/CategoriaDemoService.java` y `CategoriaDemoController.java`.
- Acoplamiento reconocido: `ProductoPersistenceAdapter.java:3` (import de `SpringDataCategoriaRepository`).

Demuéstralo

Abre `ProductoService.java` y muestra la lista de imports: ninguno empieza con `org.springframework.data`, `jakarta.persistence` ni `...infrastructure`.

Repregunta de presión

Si el servicio importa una clase concreta de PostgreSQL o un Controller, ¿qué acoplamiento aparece?

El núcleo quedaría atado a una tecnología: no se podría probar sin base ni cambiar de persistencia, y cualquier cambio en el controller o en el driver obligaría a tocar reglas de negocio. Se invertiría la dirección: el centro dependería del borde.

Pregunta 06

¿MVC y arquitectura hexagonal son lo mismo? ¿Cómo conviven en Spring Boot?

Qué responder

- No. **Spring MVC** organiza la interacción web (recibir la petición, enrutarla a un método, serializar la respuesta). **Hexagonal** organiza dependencias y fronteras de todo el sistema.
- Conviven así: Spring MVC es **un adaptador de entrada** dentro de la arquitectura hexagonal. El controller es la frontera: traduce HTTP a una llamada al puerto IN y el resultado de vuelta a HTTP.

Muéstralo en el código

- Fin de MVC / inicio del caso de uso: `ProductoController.java`, línea donde se llama `registrar.registrar(ProductoWebMapper.toDomain(request))` (método `crear`, desde la línea 46).

Repregunta de presión

Ubica exactamente dónde termina MVC y dónde comienza tu caso de uso.

MVC termina cuando el controller ya tiene el DTO validado y lo convierte a dominio (`ProductoWebMapper.toDomain`). El caso de uso comienza en la llamada `registrar.registrar(...)`, que entra por el proxy transaccional de `ProductoService.registrar` (línea 39).

Pregunta 07

En una API REST con Spring MVC, ¿dónde están Model, View y Controller?

Qué responder

- **Controller:** las clases `@RestController` (`ProductoController`, `CategoriaController`, `UbicacionController`, `InventarioController`).
- **Model:** el objeto que devuelve el método, que en mi API es un **DTO record** (`ProductoResponse`), no la entidad JPA.
- **View:** no hay plantilla HTML ni `ViewResolver`. La 'vista' es la **representación JSON** que genera Jackson a través de los `HttpMessageConverters`.

Muéstralo en el código

- `product/infrastructure/adapters/web/dto/ProductoResponse.java:7` (record).
- `ProductoController.java:27` (`@RestController`).

Repregunta de presión

¿Por qué @RestController cambia la forma tradicional de pensar la 'vista'?

Porque incluye `@ResponseBody`: el valor de retorno se escribe directamente en el cuerpo HTTP (JSON) en vez de usarse como nombre de una vista a renderizar. La presentación la hace el cliente (React, móvil), no el servidor.

Pregunta 08

¿Qué hace @SpringBootApplication y qué ocurre cuando ejecutas main()?

Qué responder

- `@SpringBootApplication` combina tres cosas: `@SpringBootConfiguration` (clase de configuración), `@EnableAutoConfiguration` (configura automáticamente según las dependencias: `DataSource`, `JPA`, `Flyway`, `Jackson`, `Tomcat`) y `@ComponentScan` (busca componentes desde el paquete de la clase hacia abajo).
- Al ejecutar `main()`: `SpringApplication.run` crea el `ApplicationContext`, escanea `com.stockflow` y registra beans (`@Service`, `@Component`, `@RestController`, `@Configuration`), crea los proxies de Spring Data, **Flyway aplica las migraciones**, Hibernate **valida el esquema** (`ddl-auto: validate`) y arranca **Tomcat embebido** en el puerto 8080.
- La ubicación importa: `StockFlowApplication` está en `com.stockflow`, la raíz, por eso detecta los cinco módulos.

Muéstralo en el código

- `StockFlowApplication.java:6` (`@SpringBootApplication`) y `:9` (`main`).
- Log al arrancar: líneas de Flyway (`Successfully applied ...`) y `Started StockFlowApplication`.

Repregunta de presión

Si colocas la clase principal en un paquete demasiado interno, ¿qué dejaría de detectarse?

Todo lo que esté fuera de ese subpaquete. Por ejemplo, si estuviera en `com.stockflow.product`, no se escanearían `category`, `location`, `inventory` ni `shared`: fallaría el arranque porque `ProductoService` necesita un bean `ConsultarCategoriaUseCase` que no existe (`NoSuchBeanDefinitionException`), y además desaparecería el `GlobalExceptionHandler`.

Pregunta 09

Explica IoC, inyección de dependencias y por qué prefieres inyección por constructor.

Qué responder

- **IoC:** no es mi código el que crea y conecta los objetos, sino el contenedor de Spring. **DI:** cada clase declara qué necesita y Spring se lo entrega.
- En StockFlow **toda** la inyección es por constructor, con campos `final` y sin `@Autowired` (con un solo constructor no hace falta).
- Ventajas: las dependencias son explícitas, el objeto nunca queda a medio construir, los campos son inmutables y se puede probar con `new` pasando falsos: `new ProductoService(fakeRepository, consultarCategoriaFalso)` en `ProductoServiceTest`.

Muéstralo en el código

- `ProductoService.java:31` (constructor con `ProductoRepositoryPort` y `ConsultarCategoriaUseCase`).
- `ProductoServiceTest.java (setUp)`: construye el servicio a mano con dependencias falsas.

Repregunta de presión

¿Qué problema tendrías si dentro del Controller haces `new MiServicio()`?

Ese objeto no sería un bean: no tendría sus dependencias inyectadas (habría que crearlas también a mano), **no pasaría por el proxy de `@Transactional`** (no habría transacción) y no se podría reemplazar por un mock en las pruebas. El controller quedaría acoplado a una implementación concreta.

Pregunta 10

Haz una auditoría de todas las anotaciones Spring/JPA de tu proyecto: clasifícalas y explica para qué sirve cada una.

Qué responder

- **Web (Spring MVC):** `@RestController`, `@RequestMapping`, `@GetMapping`, `@PostMapping`, `@PutMapping`, `@DeleteMapping`, `@PathVariable`, `@RequestParam`, `@RequestBody`, `@RestControllerAdvice`, `@ExceptionHandler`.
- **Inyección / configuración:** `@SpringBootApplication`, `@Service`, `@Component` (adaptadores de persistencia), `@Configuration`, `@Bean` (`OpenApiConfig`), `@Value` (`CorsConfig`).
- **Transacción:** `@Transactional` y `@Transactional(readOnly = true)` en los servicios.
- **Validación:** `@Valid`, `@NotNull`, `@NotBlank`, `@Size`, `@Pattern`, `@Positive`, `@PositiveOrZero`, `@Digits`.

- **Persistencia (JPA/Hibernate):** `@Entity`, `@Table`, `@Id`, `@GeneratedValue`, `@Column`, `@Enumerated`, `@ManyToOne`, `@JoinColumn`, `@OneToMany`, `@CreationTimestamp`, `@UpdateTimestamp`, `@Immutable`; en Spring Data: `@Query`, `@Param`.
- **Pruebas:** `@Test`, `@WebMvcTest`, `@MockitoBean`, `@DataJpaTest`, `@Testcontainers`, `@Container`, `@ServiceConnection`.
- No uso `@Repository`: mis repositorios son interfaces de Spring Data; Spring crea el proxy y aplica la traducción de excepciones igual.

Muéstralo en el código

- Web: `ProductoController.java`. Validación: `CrearProductoRequest.java`. JPA: `ProductoJpaEntity.java` y `CategoriaJpaEntity.java`:62. Errores: `GlobalExceptionHandler.java`:27.

Repregunta de presión

Elige tres anotaciones: ¿qué pasa si las eliminas?

`@Valid` (`ProductoController.crear`): el DTO ya no se valida; un código vacío llega al dominio, que lanza `DatoInvalidoException` → 422 en vez de 400 con errores por campo. `@Service` (`ProductoService`): no se crea el bean y el arranque falla porque el controller necesita `RegistrarProductoUseCase` (`NoSuchBeanDefinitionException`). `@JoinColumn` (`ProductoJpaEntity`:34): Hibernate infiere la columna `categoria_categoria_id`; con `validate` el arranque falla: `Schema-validation: missing column [categoria_categoria_id] in table [stockflow.producto]` (comprobado en el proyecto).

Bloque B — HTTP, MVC y flujo CRUD de punta a punta

Pregunta 11 · CRÍTICA

Explica el recorrido exacto de una solicitud desde navegador/Postman/Swagger hasta PostgreSQL y de regreso.

Qué responder

- **Ida** (ejemplo `POST /api/productos`): 1) el cliente HTTP envía la petición; 2) **Tomcat** embebido la recibe y pasa por los filtros (por ejemplo CORS); 3) el **DispatcherServlet** consulta al **HandlerMapping**, que encuentra `ProductoController.crear` por `@RequestMapping("/api/productos") + @PostMapping`; 4) **Jackson** (un `HttpMessageConverter`) convierte el JSON en el record `CrearProductoRequest`; 5) **Bean Validation** ejecuta las anotaciones por `@Valid`; 6) mi código: `ProductoWebMapper.toDomain` → `registrar.registrar(...)` → proxy transaccional → `ProductoService.registrar` → `ProductoRepositoryPort.guardar` → `ProductoPersistenceAdapter` → `repository.save` → **Hibernate** genera el `INSERT` → **JDBC** (driver de PostgreSQL) → **PostgreSQL** ejecuta y devuelve el id generado.
- **Vuelta**: resultado SQL → `ProductoJpaEntity` → `ProductoPersistenceMapper.toDomain` → `Producto` → commit de la transacción → `ProductoWebMapper.toResponse` → `ProductoResponse` → **Jackson** lo serializa → `ResponseEntity.created(location)` fija **201** y el header `Location` → el cliente recibe la respuesta.
- **Mi código**: controller, DTO, mappers, servicio, puertos, adaptador, entidad. **Del framework**: Tomcat, DispatcherServlet, HandlerMapping, Jackson, Bean Validation, el proxy de `@Transactional`, Spring Data, Hibernate y el driver JDBC.

Muéstralo en el código

- `ProductoController.java:46` → `ProductoService.java:39` → `ProductoPersistenceAdapter.java:34` → `ProductoJpaEntity.java:23`.
- SQL visible en consola: `application.yml:33` (`org.hibernate.SQL: debug`).

Demuéstralo

Ejecuta el `POST` de `requests.http` y señala en la consola el `insert into stockflow.producto (...)` que imprime Hibernate. Después muestra la fila en DataGrip.

Repregunta de presión

¿En qué punto exacto se convierte JSON en objeto Java y en qué punto un objeto Java en JSON?

JSON → Java: antes de entrar a mi método, cuando Spring resuelve el parámetro `@RequestBody CrearProductoRequest` con Jackson (`MappingJackson2HttpMessageConverter`). Si el JSON está mal formado, ni siquiera se ejecuta el controller: `HttpMessageNotReadableException` → 400. **Java → JSON**: después de que mi método devuelve `ResponseEntity<ProductoResponse>`; Spring usa el mismo converter para escribir el cuerpo.

Pregunta 12

¿Qué diferencia hay entre `@Controller` y `@RestController`? ¿Qué aporta `@RequestMapping`?

Qué responder

- `@RestController` = `@Controller` + `@ResponseBody`: lo que devuelve cada método se serializa en el cuerpo de la respuesta (JSON).
- `@RequestMapping` en la clase define la **ruta base** (`/api/productos`); los mappings por método (`@GetMapping("/{id}")`, `@PostMapping("/{id}")`) la especializan por verbo HTTP y subruta.
- Rutas reales: `/api/categorias`, `/api/productos`, `/api/ubicaciones`, `/api/inventario`, `/api/health`.

Muéstralo en el código

- `ProductoController.java:27-28` y `CategoriaController.java:29-30`.

Repregunta de presión

Si cambias `@RestController` por `@Controller` sin `@ResponseBody`, ¿qué intentaría hacer Spring?

Tomaría el valor devuelto como **modelo** y buscaría una **vista** (plantilla) con el nombre derivado de la ruta. Como no hay plantillas, esos endpoints fallarían por no poder resolver la vista. Los métodos que devuelven `ResponseEntity` seguirían escribiendo el cuerpo, pero `listar` (que devuelve `List<...>`) dejaría de funcionar.

Pregunta 13

Diferencia `@PathVariable`, `@RequestParam` y `@RequestBody` con endpoints reales.

Qué responder

- `@PathVariable`: un segmento de la ruta que **identifica** el recurso. `GET /api/productos/{id}`. Spring convierte "5" a `Long`; si no puede (`/abc`), `MethodArgumentTypeMismatchException` → 400.
- `@RequestParam`: parámetros de la **query string**, normalmente filtros opcionales. `GET /api/categorias?nombre=limp`, `GET /api/ubicaciones?tipo=DEPOSITO` (Spring convierte el texto al enum), `GET /api/inventario/kardex/1?pagina=0&tamano=20`.
- `@RequestBody`: el **cuerpo** de la petición (JSON) convertido por Jackson a un record. Se usa en POST y PUT. Un GET no se diseña alrededor de un cuerpo.

Muéstralo en el código

- `CategoriaController.java:55` (`@RequestParam`), `:62` (`@PathVariable`), `:47` (`@RequestBody`).
- `InventarioController.kardex`: `@PathVariable` + dos `@RequestParam` con `defaultValue`.

Repregunta de presión

Diseña una ruta para buscar los hijos de un padre por estado: ¿qué va en el path y qué en la query string?

El padre identifica el recurso, va en el **path**; el estado es un filtro, va en la **query**: `GET /api/productos/categoria/{categoriaId}?activo=true`. Es el mismo criterio que ya uso en `/api/inventario/stock?productoId=&ubicacionId=&soloBajoMinimo=`.

Pregunta 14

¿Swagger, Postman y un navegador 'pasan por caminos distintos' dentro de tu backend?

Qué responder

- No. Los tres son **clientes HTTP**. Si llaman al mismo endpoint, dentro del backend recorren exactamente el mismo camino: Tomcat → DispatcherServlet → controller → caso de uso → persistencia.
- **Swagger UI** (/swagger-ui.html) lee la especificación OpenAPI que springdoc genera desde mis controllers (/v3/api-docs) y permite probarlos. **Postman** o **requests.http** construyen la petición a mano. Un **navegador** hace GET directos o, desde React, peticiones con **fetch**.
- Swagger es documentación y prueba, **no** es el backend ni contiene lógica de negocio.

Muéstralo en el código

- `shared/web/OpenApiConfig.java:15` y la dependencia `springdoc-openapi-starter-webmvc-ui` en `pom.xml`.
- `requests.http` (cliente HTTP de IntelliJ).

Demuéstralo

Abre `http://localhost:8080/swagger-ui.html`, ejecuta `GET /api/productos/{id}` y muestra que en la consola aparece el mismo `select` que con `requests.http`.

Repregunta de presión

Si funciona en Swagger pero falla desde React, ¿qué revisas primero?

Primero **la URL y el puerto** (¿apunta a :8080 y a /api/...?), luego **CORS** (la consola del navegador muestra el bloqueo; Swagger está en el mismo origen y no lo sufre), después **método y headers** (`Content-Type: application/json`), el **payload** (nombres de campos y tipos iguales al DTO) y por último la serialización de la respuesta. Lo confirmo en la pestaña Network comparando con la petición de Swagger.

Pregunta 15 · CRÍTICA

POST: explica línea por línea qué ocurre al crear una EntidadPadre o EntidadHija.

Qué responder

- Uso la entidad hija, **Producto** (`POST /api/productos`):
- 1) `@PostMapping` en `ProductoController.crear`. 2) Jackson crea el record `CrearProductoRequest`. 3) `@Valid` ejecuta `@NotNull` `categoriaId`, `@NotBlank` + `@Pattern` en `codigo`, `@PositiveOrZero` `stockMinimoDefault`, `@Digits` en el precio → si falla, **400** con `fieldErrors`. 4) `ProductoWebMapper.toDomain` llama a `Producto.nuevo`: normaliza el código a mayúsculas, pone **UNIDAD** si no llega unidad, el precio queda con 2 decimales y `activo = true`.
- 5) `ProductoService.registrar (@Transactional)`: busca la categoría con el **puerto IN del módulo padre** (`ConsultarCategoriaUseCase`) → **404** si no existe, **422** si está inactiva; verifica el código único con `existePorCodigo` → **409** (RN-06). 6) `repositoryPort.guardar` → `ProductoPersistenceAdapter.guardar`.
- 7) **Relación ManyToOne**: `categoriaRepository.getReferenceById(categoriaId)` crea un **proxy** con sólo el id (sin SELECT); `ProductoPersistenceMapper.toJpa` lo asigna al campo `categoria`. 8) `repository.save(entity)`: como el id es null, es una entidad nueva → `persist`. Con **IDENTITY**, Hibernate ejecuta el **INSERT** en ese mismo momento (no puede esperar al commit, porque necesita el id) y PostgreSQL devuelve el `producto_id` generado. La FK `fk_producto_categoria` vuelve a proteger el dato en la base.

- 9) Se mapea a dominio y luego a `ProductoResponse`; al salir del servicio se hace **commit**. 10) El controller arma la URI `/api/productos/{id}` y devuelve **201 Created** con el header `Location`.
- **Persistir vs. actualizar:** el POST crea una entidad nueva (transient → managed con `persist`). El PUT carga una entidad existente (`findById`, ya `managed`) y la modifica: Hibernate genera un `UPDATE` por dirty checking, sin llamar a `save()`.

Muéstralo en el código

- `ProductoController.java:46`, `ProductoWebMapper.java:13`, `ProductoService.java:39-48`, `ProductoPersistenceAdapter.java:34-38`, `ProductoPersistenceMapper.java:12`.
- FK en la base: `db/migration/V1__creacion_completa_stockflow.sql:103` (`fk_producto_categoria`).

Repregunta de presión

Si el id del padre recibido no existe, ¿dónde lo detectas y qué status devuelves?

En el **caso de uso**: `consultarCategoriaUseCase.buscarPorId(...).orElseThrow(() -> new CategoriaNoEncontradaException(...))` (`ProductoService.java:40`) → `GlobalExceptionHandler` la traduce a **404**. Si alguien se saltara esa validación, la FK de PostgreSQL rechazaría el INSERT → `DataIntegrityViolationException` → 409. Dos capas, a propósito.

Pregunta 16 · CRÍTICA

GET por id: explica todo el flujo, incluido el caso en que el registro no exista.

Qué responder

- `@GetMapping("/{id}") + @PathVariable Long id` → `consultar.buscarPorId(id)` (puerto IN, `@Transactional(readOnly = true)`) → `repositoryPort.buscarPorId` → `repository.findById(id)` → `SELECT ... WHERE producto_id = ?` → `Optional<ProductoJpaEntity>` → `.map(ProductoPersistenceMapper::toDomain)` → `Optional<Producto>`.
- En el controller: `.orElseThrow(() -> new ProductoNoEncontradoException(id))`. Si existe: `ProductoResponse` y **200**. Si no: la excepción (subclase de `RecursoNoEncontradoException`) llega al `GlobalExceptionHandler` → **404** con `ApiErrorResponse` (timestamp, status, message, path).
- **Relaciones:** la categoría es `@ManyToOne(fetch = LAZY)`, así que se carga como **proxy**. El mapper sólo lee `getCategoria().getId()`, que el proxy ya conoce: **no se ejecuta un segundo SELECT**. Al pedir una categoría, su colección `productos` (`@OneToMany`, `LAZY` por defecto) tampoco se carga, porque nadie la recorre.

Muéstralo en el código

- `ProductoController.java:54-57` (`orElseThrow`), `ProductoService.java:85`, `ProductoPersistenceAdapter.java:68-69`, `ProductoPersistenceMapper.java:30` (`e.getCategoria().getId()`).
- Prueba: `ProductoControllerTest.get404SiNoExiste`.

Repregunta de presión

¿Qué diferencia práctica hay entre `findById(id).get()` y `orElseThrow(...)`?

`get()` sobre un `Optional` vacío lanza `NoSuchElementException`, que mi handler trataría como error no previsto → **500**, como si fuera un fallo del servidor. `orElseThrow` expresa el caso de uso: 'si no existe, es un recurso no encontrado' → **404** con un mensaje claro.

Pregunta 17

GET de listado: ¿qué cambia respecto al GET por id y qué riesgos aparecen con relaciones?

Qué responder

- Devuelve una **colección**: se mapea cada elemento con `stream().map(...)` y responde **200**, incluso con lista vacía (vacío no es error).
- Ejemplos: `GET /api/categorias?nombre=` (consulta derivada `findByNombreContainingIgnoreCaseOrderByCodigoAsc`) y `GET /api/productos/categoria/{id}` (`findByCategoria_IdOrderByCodigoAsc`).
- **Riesgos**: traer demasiados registros (memoria, tiempo, tamaño de respuesta) y el **N+1** si al mapear se recorren relaciones LAZY. Por eso: devuelvo DTO planos (`categoriaId`, no el objeto categoría), los listados no recorren `categoria.getProductos()` y el **kardex**, que es lo que más crece, está **paginado**.

Muéstralo en el código

- Paginación: `InventarioController.kardex` → `SpringDataStockRepository.java:90` (`Page<KardexFila> buscarKardex(..., Pageable pageable)`).
- Respuesta paginada: `PaginaResponse` (`contenido, pagina, tamano, totalElementos, totalPaginas`).

Demuéstralo

`GET /api/inventario/kardex/1?pagina=0&tamano=1` → `totalElementos: 2, totalPaginas: 2`. En la consola aparecen el `select ... limit/offset` y el `select count(*)`.

Repregunta de presión

Si tienes 10.000 padres con 100 hijos cada uno, ¿por qué un `findAll()` con todo anidado es un desastre?

Serían 1.000.000 de objetos en memoria y un JSON enorme. Si la colección es LAZY y el mapper la recorre, son $1 + 10.000$ consultas (N+1); si fuera EAGER, la carga sería gigante siempre. La solución es paginar, devolver DTO sin las colecciones anidadas y, si hace falta el detalle, un endpoint específico (`/categoria/{id}`) o una consulta con `JOIN FETCH` o proyección.

Pregunta 18 · CRÍTICA

PUT: explica el flujo completo de una actualización y por qué PUT debe ser idempotente.

Qué responder

- `PUT /api/productos/{id}` con `ActualizarProductoRequest` (representación **completa**, sin `id`; `activo`, `unidadMedida` y `stockMinimoDefault` son obligatorios).
- 1) `@Valid` → 400 si falta algo. 2) `ProductoService.actualizar(id, cambios)` (`@Transactional`): busca el producto → **404 si no existe (el PUT no crea)**; valida la categoría destino → 404 si no existe, **422** si cambia a una categoría inactiva; unicidad **excluyendo el propio registro** con `existePorCodigoEnOtro` → `existsByCodigoAndIdNot` (si no, editar sin cambiar el código daría un 409 falso). 3) Construye el dominio con el **id de la ruta** (la identidad no la decide el cuerpo).
- 4) `ProductoPersistenceAdapter.actualizar`: `findById` deja la entidad **managed**; `entity.actualizar(...)` cambia sus campos; `repository.flush()` → Hibernate compara con el estado original (**dirty checking**) y genera el `UPDATE`, **sin llamar a `save()`**. El `flush` adelanta a ese punto un posible error de la base (UNIQUE o CHECK). 5) **200** con el recurso actualizado.
- **Idempotencia**: enviar el mismo PUT 1 o 10 veces deja el recurso en el mismo estado final. Lo pruebo en `ProductoServiceTest.putEsIdempotente`.

- **PUT vs. PATCH:** PUT reemplaza la representación completa; PATCH aplicaría cambios parciales. No implementé PATCH.

Muéstralo en el código

- `ProductoController.java:69`, `ProductoService.java:53-68`, `ProductoPersistenceAdapter.java:42-51`, `ProductoJpaEntity.java:85` (actualizar).
- `SpringDataProductoRepository.java:13` (`existsByCodigoAndIdNot`).
- Pruebas: `ProductoServiceTest` (8 casos de PUT), `ProductoControllerTest.put400SiFaltaActivo`.

Repregunta de presión

¿Qué error aparece si construyes una entidad nueva con el id recibido y haces `save()` sin verificar existencia?

Como el id no es null, `save()` hace un `merge`. Con un id que no existe, en **Hibernate 6.6 (la versión de mi proyecto)** el merge lanza `ObjectOptimisticLockingFailureException: Row was updated or deleted by another transaction (or unsaved-value mapping was incorrect)`, que en mi API terminaría como **500**. Lo comprobé en el proyecto. En versiones anteriores de Hibernate se insertaba en silencio una fila nueva **con otro id**, peor todavía. Por eso el servicio verifica la existencia y responde 404.

Pregunta 19 · CRÍTICA

DELETE: explica el flujo completo y qué ocurre con los hijos en una relación uno-a-muchos.

Qué responder

- `@DeleteMapping("/{id}")` → `EliminarCategoriaUseCase.eliminar(id)` (`@Transactional`) → si no existe, **404** → `CategoriaPersistenceAdapter.eliminar: repository.deleteById(id) + repository.flush()` para que el **DELETE** se ejecute ahí mismo → **204 No Content** sin cuerpo.
- **Hijos:** las FK de V1 **no tienen CASCADE** y mi mapping **no usa cascade ni orphanRemoval**. Si la categoría tiene productos, PostgreSQL rechaza el **DELETE** por `fk_producto_categoria`; el adaptador captura `DataIntegrityViolationException` y lanza `CategoriaConProductosException` → **409**. Lo mismo con un producto que tiene stock o movimientos → `ProductoEnUsoException` → 409.
- Para retirar algo que tiene historia se usa **PUT con `activo = false`** (ADR-004).

Muéstralo en el código

- `CategoriaController.java:77-80`, `CategoriaService.java:60`, `CategoriaPersistenceAdapter.java:44-49` (`try/catch`).
- Pruebas contra PostgreSQL real: `PersistenciaPostgresTest.eliminarCategoriaConProductosLanzaConflictoDeDominio y ...eliminarProductoConStock...`

Demuéstralo

DELETE /api/categorias/1 → 409 (CAT-OFI tiene productos). Crea una categoría nueva y bórrala → 204. Bórrala otra vez → 404.

Repregunta de presión

Si la base rechaza el DELETE por FK, ¿por qué no 'arreglarlo' poniendo CascadeType.ALL?

Porque **ALL** incluye **REMOVE**: borrar una categoría borraría todos sus productos, y con ellos su stock y movimientos, es decir, el **kardex** (RN-01, trazabilidad). Además las FK no tienen CASCADE en la base. Es una decisión de **negocio**, no un truco para que deje de fallar: en StockFlow un producto con historia no debe desaparecer.

Pregunta 20

Explica los códigos HTTP 200, 201, 204, 400, 404, 409 y 500 en el contexto de tu API.

Qué responder

- **200 OK**: GET y PUT exitosos (GET `/api/productos/1`). **201 Created**: POST que crea, con header `Location`. **204 No Content**: DELETE exitoso, sin cuerpo.
- **400 Bad Request**: la petición está mal formada (@Valid falla, JSON inválido, `unidadMedida: "BARRIL"`, `/api/productos/abc`). **404 Not Found**: el recurso o el padre no existe. **409 Conflict**: choca con datos existentes (código duplicado RN-06, DELETE bloqueado por FK, segundo almacén central). **500**: error no previsto; el cliente recibe sólo 'Error interno del servidor' y el detalle va al log.
- Uso además **422 Unprocessable Entity**: la petición está bien formada pero viola una regla de negocio (categoría inactiva, dato inválido en el dominio, página fuera de rango).
- Nunca devuelvo 200 con un error dentro del cuerpo: el código lo decide `GlobalExceptionHandler` según el tipo de excepción.

Muéstralo en el código

- `shared/web/GlobalExceptionHandler.java:32-92` (un `ExceptionHandler` por tipo).
- `ProductoControllerTest` verifica 201, 400, 404, 409, 422, 204 y 500.
- Tabla completa en `docs/03-decisions/ADR-006-contrato-de-errores.md`.

Repregunta de presión

Dame un ejemplo de tu proyecto donde 409 sea más correcto que 400.

`POST /api/productos` con código `PRD-OFI-001`: el JSON es **válido** (por eso no es 400), pero ese código ya existe en la base (`uq_producto_codigo`, RN-06). El problema no está en la forma de la petición, sino en el **conflicto con el estado actual** del servidor: 409.

Pregunta 21

¿Qué hace @Valid y en qué momento se ejecutan @NotNull, @NotBlank, @Size u otras validaciones?

Qué responder

- `@Valid` en el parámetro `@RequestBody` activa **Bean Validation** (Hibernate Validator) **después** de que Jackson crea el record y **antes** de ejecutar el cuerpo del método del controller. Si falla, se lanza `MethodArgumentNotValidException` y mi handler responde **400** con un mapa `fieldErrors` (campo → mensaje).
- Es validación **estructural**: obligatorio, tamaño, formato, rango. Las reglas de negocio (código único, categoría activa) van en el servicio.
- `@NotNull` acepta "" y " "; `@NotBlank` (sólo para texto) rechaza null, vacío y sólo espacios. Por eso `codigo` y `nombre` usan `@NotBlank` y `categoriaId` (un `Long`) usa `@NotNull`.

Muéstralo en el código

- `CrearProductoRequest.java:16-40`; handler en `GlobalExceptionHandler.java:32`.
- Prueba: `ProductoControllerTest.post400ConErroresPorCampoYSinLlegarAlCasoDeUso` (verifica que el caso de uso **no** se llama).

Repregunta de presión

Un correo con formato correcto pero ya registrado: ¿es problema de @Email o regla de negocio?

Regla de negocio. @Email sólo verifica la forma; saber si ya existe requiere consultar la base, y eso lo hace el caso de uso (→ 409), como mi `existePorCodigo`. Al final, el UNIQUE de PostgreSQL es la última barrera.

Pregunta 22

¿Cómo centralizas errores con @RestControllerAdvice y @ExceptionHandler?

Qué responder

- `GlobalExceptionHandler` está anotado con `@RestControllerAdvice`: intercepta las excepciones de **todos** los controllers. Cada `@ExceptionHandler` traduce un tipo de excepción a un status y construye siempre el mismo record `ApiErrorResponse` (`timestamp`, `status`, `error`, `message`, `path`, `fieldErrors`).
- Uso **jerarquías** para no listar cada excepción: todo `RecursoNoEncontradoException` → 404, `ConflictoNegocioException` → 409, `ReglaNegocioException` → 422. `CodigoProductoDuplicadoException` extends `ConflictoNegocioException`, por ejemplo.
- Último recurso: `@ExceptionHandler(Exception.class)` → 500 con mensaje genérico y el detalle sólo en el log. Los errores propios de Spring (405, 404 de ruta) respetan su status.
- Resultado: **ningún controller tiene try/catch**.

Muéstralo en el código

- `shared/web/GlobalExceptionHandler.java:27`; `shared/web/ApiErrorResponse.java:7`; `shared/domain/exception/`.

Repregunta de presión

¿Qué ganas frente a capturar Exception en cada endpoint y devolver 500 manualmente?

Un **contrato único** de error para web y móvil, controllers limpios, un solo lugar para cambiar el formato y **códigos correctos**: no todo es 500; un duplicado es 409 y un id inexistente es 404. Además el 500 no filtra detalles internos (lo prueba `errorNoControlado500SinExponerElDetalleInterno`).

Pregunta 23

¿Para qué sirve @Transactional y dónde debería estar el límite de la transacción?

Qué responder

- Garantiza **atomicidad**: todo lo que hace el método se confirma junto (commit) o se deshace junto (rollback) si se lanza una `RuntimeException`, y todas mis excepciones de negocio lo son.
- El límite está en el **caso de uso** (`ProductoService.registrar`, `actualizar`, `eliminar`), porque es la operación completa del negocio: validar el padre, verificar unicidad y guardar. No va en el controller (HTTP) ni en el repositorio (demasiado pequeño).
- Las consultas usan `@Transactional(readOnly = true)`.

- Relación con el **persistence context**: mientras dura la transacción, las entidades cargadas están *managed*; por eso el PUT se guarda por **dirty checking** al hacer flush/commit.

Muéstralo en el código

- `ProductoService.java:38, 52, 74 (@Transactional)` y `:84, 90 (readOnly = true)`.
- `application.yml:21 open-in-view: false`: la sesión de Hibernate no sigue abierta durante la vista; las entidades no se usan fuera del servicio.

Repregunta de presión

Si guardas el padre y luego falla el guardado de un hijo, ¿cómo evitas quedar con datos a medias?

Haciendo las dos operaciones **dentro del mismo método @Transactional**: si el hijo falla con una `RuntimeException`, Spring hace rollback y el padre tampoco queda guardado. Es lo que demuestro en SQL con el flujo crítico de `datagrip/03_verificacion_y_pruebas.sql` (sección 8: `BEGIN ... COMMIT`; si un paso falla, `ROLLBACK`).

Pregunta 24

¿Qué es CORS y por qué puede fallar React en el navegador aunque Postman funcione?

Qué responder

- CORS (Cross-Origin Resource Sharing) es una política **del navegador**: una página de un origen sólo puede leer respuestas de otro origen si el servidor lo autoriza con headers `Access-Control-Allow-*`.
- Origen = **esquema + host + puerto**. Para métodos como PUT o DELETE, el navegador manda antes una petición `OPTIONS (preflight)`.
- Postman no es un navegador y no aplica CORS; por eso 'funciona'. No tiene nada que ver con autenticación ni con la base de datos.
- En StockFlow: `CorsConfig` permite `/api/**` desde los orígenes de `stockflow.cors.allowed-origins` (por defecto `http://localhost:5173`, Vite), con GET, POST, PUT, DELETE y OPTIONS, y expone el header `Location`. Se configura con la variable `CORS_ALLOWED_ORIGINS`; nunca con `*`.

Muéstralo en el código

- `shared/web/CorsConfig.java:13-26`; `application.yml:29`.
- `requests.http`: sección CORS con un preflight permitido (200 + `Access-Control-Allow-Origin`) y uno no permitido (403).

Repregunta de presión

Backend en :8080 y React en :5173: ¿por qué son orígenes distintos si los dos están en localhost?

Porque el origen incluye el **puerto**: `http://localhost:5173` y `http://localhost:8080` difieren en el puerto, así que para el navegador son orígenes distintos aunque sea la misma máquina.

Bloque C — JPA, Hibernate, cardinalidad y PostgreSQL

Pregunta 25

¿Qué hacen `@Entity` y `@Table` y qué significa que una clase sea gestionada por JPA?

Qué responder

- `@Entity` marca la clase como **persistente**: Hibernate puede convertir sus instancias en filas y al revés. `@Table(name = "producto", schema = "stockflow")` dice exactamente a qué tabla y esquema corresponde, porque el nombre de la clase (`ProductoJpaEntity`) no coincide con el de la tabla.
- Tres cosas distintas: la **clase Java** (`ProductoJpaEntity`), la **entidad persistente** (una instancia *managed* dentro del persistence context) y la **tabla física** (`stockflow.producto`, creada por Flyway en V1).
- 'Gestionada' significa que, dentro de una transacción, Hibernate sigue esa instancia: detecta sus cambios y los sincroniza con la base.

Muéstralo en el código

- `product/infrastructure/adapters/out/persistence/entity/ProductoJpaEntity.java:23-24`.
- Tabla: `db/migration/V1__creacion_completa_stockflow.sql:90` (`CREATE TABLE producto`).

Repregunta de presión

Si renombras la tabla en PostgreSQL pero no actualizas el mapping, ¿qué ocurre?

Con `ddl-auto: validate` la aplicación **no arranca**: Hibernate valida el esquema al iniciar y lanza un error `Schema-validation: missing table ...`. Es lo deseable: el error aparece al arrancar, no en producción en medio de una petición. Con `none` arrancaría y fallaría en el primer SQL.

Pregunta 26

Explica `@Id` y `@GeneratedValue`. ¿Cómo se genera tu identificador en PostgreSQL?

Qué responder

- `@Id` marca la clave primaria (`producto_id`). `@GeneratedValue(strategy = GenerationType.IDENTITY)` delega la generación a la base.
- En PostgreSQL la columna es `BIGINT GENERATED BY DEFAULT AS IDENTITY`: PostgreSQL asigna el siguiente valor al hacer el INSERT. Las dos definiciones coinciden, y `ddl-auto: validate` lo comprueba.
- El id es la **identidad persistente**: no es un dato editable. Por eso no viaja en `CrearProductoRequest` ni en el cuerpo del PUT; en el PUT llega por la ruta.

Muéstralo en el código

- `ProductoJpaEntity.java:27-29`; `V1__creacion_completa_stockflow.sql:76` (`GENERATED BY DEFAULT AS IDENTITY`).

Repregunta de presión

¿En qué momento tiene id la entidad nueva? ¿Depende de la estrategia?

Sí depende. Con **IDENTITY** el id existe **después del INSERT**, y Hibernate ejecuta ese INSERT en el mismo `save()/persist()` (no puede esperar al commit). Con **SEQUENCE**, Hibernate pide el siguiente valor a la secuencia antes del INSERT, así que el id existe al hacer `persist` y el INSERT puede postergarse hasta el flush (eso permite el *batching*).

Pregunta 27

¿Qué puede expresar @Column y qué diferencia hay entre esa configuración y una restricción real en PostgreSQL?

Qué responder

- `@Column` indica el **nombre** de la columna (`codigo`), `nullable`, `unique`, `length` (40) y, para decimales, `precision/scale` (`precio_referencial`: 12, 2).
- Esa configuración es **metadato de Java**: sirve para que Hibernate valide o genere DDL. En StockFlow el esquema **no** lo genera Hibernate (`validate`): las restricciones reales están en V1 (`NOT NULL`, `uq_producto_codigo`, `ck_producto_unidad_medida`, `ck_producto_precio`). **La base hace cumplir la integridad.**
- No confundir con Bean Validation (`@NotBlank`, `@Size` en el DTO): esa valida la entrada HTTP; `@Column` describe el mapeo; la constraint de PostgreSQL es la garantía final.

Muéstralo en el código

- `ProductoJpaEntity.java:37` (`length = 40, unique = true`), `:53` (`precision = 12, scale = 2`).
- Constraints reales: `V1__creacion_completa_stockflow.sql:103-108`.

Repregunta de presión

Si @Column(nullable = false) dice una cosa y la tabla permite NULL, ¿quién tiene la última palabra?

PostgreSQL. `validate` comprueba que las columnas existan y que el tipo sea compatible, no la nulabilidad. Al ejecutar el SQL, la base aplica sus propias reglas: si la columna admite NULL, lo acepta. Por eso las reglas importantes van como constraints en la migración.

Pregunta 28 · CRÍTICA

Representa en tablas la cardinalidad uno-a-muchos de tus dos entidades.

Qué responder

- **categoría** (padre): `categoria_id PK` · `codigo UQ` · `nombre UQ` · `descripcion` · `activo`.
- **producto** (hijo): `producto_id PK` · `categoria_id FK` → `categoria.categoria_id` · `codigo UQ` (RN-06) · `nombre` · `unidad_medida CHECK` · `stock_minimo_default CHECK ≥ 0` · `precio_referencial` · `activo`.
- La FK se llama `fk_producto_categoria` y la columna `categoria_id` es **NOT NULL**: no hay producto sin categoría. Muchas filas de `producto` pueden apuntar a la misma `categoria_id`: eso es el 1:N.
- Ejemplo de la semilla: `CAT-OFI` (id 1) tiene `PRD-OFI-001` y `PRD-OFI-002`.

Muéstralo en el código

- `V1_creacion_completa_stockflow.sql:75` (categoria), `:90-103` (producto y su FK).
- DER: `docs/04-model/der-logico-v0.1.md` (Mermaid). Consulta con JOIN: `datagrip/03_verificacion_y_pruebas.sql`, sección 6.

Demuéstralo

En DataGrip, ejecuta el JOIN de la sección 6 de `03_verificacion_y_pruebas.sql` y señala que `fk_categoria_id` coincide con `pk_categoria_id`.

Repregunta de presión

¿En qué tabla está físicamente la FK y por qué no en la tabla padre?

En **producto**, el lado N. Una celda sólo guarda un valor: cada producto apunta a **una** categoría, pero una categoría tiene **muchos** productos, y no se puede guardar una lista en una columna sin violar la primera forma normal.

Pregunta 29 · CRÍTICA

Explica @OneToMany y @ManyToOne usando exactamente tu relación.

Qué responder

- `@ManyToOne` está en el hijo: `ProductoJpaEntity.categoria`. 'Muchos productos → una categoría'. Representa la **FK** `categoria_id`.
- `@OneToMany` está en el padre: `CategoriaJpaEntity.productos` = 'una categoría → muchos productos'. Representa la **colección** vista desde el padre; no agrega ninguna columna.
- Mi relación es **bidireccional**: los dos lados existen en Java. El `@OneToMany` es el **lado inverso**, de sólo lectura: sin cascade, sin orphanRemoval y sin uso en los mappers (ADR-003).
- **Cardinalidad Java vs. relacional**: en la base hay una sola FK en `producto`; en Java, la misma relación se ve desde dos objetos. El dominio (`Producto`) ni siquiera tiene el objeto: guarda sólo `categoriaId`.

Muéstralo en el código

- `ProductoJpaEntity.java:33-34` (`@ManyToOne(fetch = LAZY, optional = false) + @JoinColumn`).
- `CategoriaJpaEntity.java:62` (`@OneToMany(mappedBy = "categoria")`) y `:93` (`getProductos()` inmodificable).
- Decisión: `docs/03-decisiones/ADR-003-relacion-1n-y-lado-inverso.md`.

Repregunta de presión

Si sólo mantuvieras @ManyToOne en el hijo, ¿la relación seguiría existiendo en la base? ¿Qué perderías en Java?

Sí, seguiría existiendo completa: la relación la define la **FK** en la tabla, no la anotación. En Java sólo perdería la navegación de la categoría a sus productos (`categoria.getProductos()`). De hecho, mi proyecto empezó así (unidireccional) y la API funcionaba igual; el lado inverso no cambió ningún endpoint.

Pregunta 30 - CRÍTICA

¿Cuál es el lado propietario de la relación y qué significan `mappedBy` y `@JoinColumn`?

Qué responder

- **Lado propietario:** `ProductoJpaEntity.categoria`, el `@ManyToOne`. Es el **único** que Hibernate usa para escribir la FK: el valor de `categoria_id` sale de ese campo.
- `@JoinColumn(name = "categoria_id")`: el nombre de la **columna SQL** de la FK en la tabla `producto`.
- `mappedBy = "categoria"`: está en el lado inverso (`CategoriaJpaEntity.productos`) y apunta al nombre del **atributo Java** del lado propietario (`ProductoJpaEntity.categoria`), **no** al de la columna. Significa: 'esta colección no tiene columna propia; la mapea el campo `categoria` del otro lado'.
- Consecuencia: agregar o quitar productos de `categoria.getProductos()` **no cambiaría la base**; para cambiar la categoría de un producto se modifica `producto.categoria` (mi PUT de producto). En mi código la lista es además inmodificable.
- **Nombres exactos:** campo `mappedBy = "categoria"`; join column = `"categoria_id"`.

Muéstralo en el código

- `ProductoJpaEntity.java:33-35` (lado propietario).
- `CategoriaJpaEntity.java:49-63` (lado inverso, con el comentario que lo explica).
- Prueba: `PersistenciaPostgresTest.ladoInversoMappedByLeeLosProductosATravesDeLaFkYEsLazy`.

Demuéstralo

Abre las dos entidades lado a lado y señala que el texto de `mappedBy ("categoria")` es igual al **nombre del campo** de `ProductoJpaEntity`, mientras que el texto de `@JoinColumn ("categoria_id")` es igual al **nombre de la columna** en V1.

Repregunta de presión

Si escribes `mappedBy` con el nombre de la columna SQL en vez del atributo Java, ¿qué error esperas?

Un error **al arrancar**, no en tiempo de ejecución: Hibernate no encuentra ese atributo al construir el `EntityManagerFactory`. Lo comprobé en el proyecto: `AnnotationException: Collection '...CategoriaJpaEntity.productos' is 'mappedBy' a property named 'categoria_id' which does not exist in the target entity '...ProductoJpaEntity'`. Y si **quito** `mappedBy`, Hibernate cree que hay dos lados propietarios y busca una tabla intermedia: `Schema-validation: missing table [categoria_productos]`.

Pregunta 31

Explica `cascade` y cada `CascadeType` que realmente utilizas. ¿Por qué `CascadeType.ALL` puede ser peligroso?

Qué responder

- Cascade **propaga operaciones** de una entidad a sus relacionadas: `PERSIST` (guardar el padre guarda los hijos nuevos), `MERGE`, `REMOVE` (borrar el padre borra los hijos), `REFRESH`, `DETACH`. `ALL` las incluye todas.
- **En StockFlow no uso ningún `CascadeType`, a propósito** (ADR-004): `@OneToMany(mappedBy = "categoria")` no tiene `cascade` y las FK de V1 tampoco tienen `ON DELETE CASCADE`.
- `ALL` es peligroso porque incluye `REMOVE`: borraría hijos que tienen vida propia. Un producto tiene stock, movimientos y kardex.

Muéstralo en el código

- `CategoriaJpaEntity.java:62` (sin `cascade`).
- Efecto observable: `DELETE /api/categorias/1` → 409 (`CategoriaPersistenceAdapter.java:49`).

Repregunta de presión

¿Tendría sentido borrar automáticamente todos los hijos al borrar el padre en tu dominio?

No, por **negocio**: un producto no deja de existir porque se reorganice el catálogo; tiene existencias, movimientos auditados y forma parte del kardex (RN-01). Lo correcto es impedir el borrado (409) y desactivar la categoría con `activo = false`, o mover sus productos a otra.

Pregunta 32

¿Qué hace `orphanRemoval=true` y en qué se diferencia de `CascadeType.REMOVE`?

Qué responder

- `orphanRemoval = true` borra un hijo cuando **deja de pertenecer** a la colección del padre (queda 'huérfano'), **aunque el padre siga existiendo**.
- `CascadeType.REMOVE` borra los hijos **sólo cuando se borra el padre**.
- Tiene sentido cuando el hijo **no tiene ciclo de vida propio** (las líneas de un pedido, por ejemplo `item_solicitud` respecto de `solicitud`).
- **En StockFlow no lo uso**: un producto no 'muere' por salir de una categoría; cambia de categoría actualizando su FK. Además mi colección es de sólo lectura.

Muéstralo en el código

- `CategoriaJpaEntity.java:62` (sin `orphanRemoval`). Cambio de categoría: `PUT /api/productos/{id}` con otro `categoriaId` (`ProductoService.java:53`).

Repregunta de presión

Si quitas un hijo de la lista del padre pero no borras el padre, ¿qué esperas con y sin `orphanRemoval`?

Con `orphanRemoval`: al hacer flush, Hibernate ejecuta `DELETE FROM producto WHERE producto_id = ?`. **Sin `orphanRemoval`** (mi caso): no pasa nada en la base, porque la colección es el lado inverso y la FK la controla `producto.categoria`. Como la FK es NOT NULL, tampoco podría dejarse en null.

Pregunta 33 · CRÍTICA

Explica `FetchType.LAZY` y `EAGER`, incluyendo los defaults de `@OneToMany` y `@ManyToOne`.

Qué responder

- **LAZY**: la relación se carga recién cuando se accede a ella (Hibernate pone un *proxy* o una colección sin inicializar). **EAGER**: se carga junto con la entidad, siempre.
- **Defaults de JPA**: `@OneToMany` → **LAZY**; `@ManyToOne` → **EAGER**.
- En mi proyecto: `@ManyToOne(fetch = FetchType.LAZY)` está **configurado explícitamente** en `ProductoJpaEntity` (cambia el default); `@OneToMany` en `CategoriaJpaEntity` usa su default LAZY.
- 'La relación existe' (hay FK) no es lo mismo que 'la relación ya está cargada'. Una relación LAZY accedida **fuera** de la transacción lanza `LazyInitializationException`. En mi código eso no pasa: los mappers trabajan dentro del servicio y sólo leen `getCategoria().getId()`, que el

proxy conoce sin consultar.

Muéstralo en el código

- `ProductoJpaEntity.java:33; CategoriaJpaEntity.java:62.`
- Prueba: `PersistenciaPostgresTest.ladoInverso...EsLazy` verifica `isLoading(oficina, "productos") == false` antes de recorrerla y `true` después.

Repregunta de presión

¿Por qué convertir todo a EAGER para evitar LazyInitializationException crea otros problemas?

Porque carga datos que la mayoría de las operaciones no necesitan, en **cada** consulta: más memoria, más SQL, más tiempo, y con colecciones puede generar consultas en cascada o productos cartesianos. Tapa el síntoma en vez de corregir el límite de la transacción o la consulta (fetch join, DTO).

Pregunta 34

En una relación bidireccional, ¿por qué conviene mantener sincronizados ambos lados en memoria?

Qué responder

- La base sólo mira el **lado propietario** (`producto.categoria`). Pero en memoria hay dos objetos: si se asigna la categoría al producto y no se agrega el producto a `categoria.productos` (o al revés), el **grafo de objetos queda incoherente** dentro de la misma transacción. La lista mostraría datos viejos y una validación o respuesta basada en ella fallaría.
- Por eso, cuando la colección se **modifica**, se usan métodos helper que actualizan los dos lados a la vez.
- **En mi proyecto** el lado inverso es de **sólo lectura** (`getProductos()` devuelve una lista inmodificable y no hay setters): nadie puede desincronizarlo, así que no hacen falta helpers. Si algún día se modificara, habría que agregarlos.

Muéstralo en el código

- `CategoriaJpaEntity.java:93 (Collections.unmodifiableList); prueba PersistenciaPostgresTest.ladoInversoEsDeSoloLectura.`

Repregunta de presión

Escribe verbalmente qué debería hacer `addHijo(hijo)` paso a paso.

`addProducto(producto)`: 1) agregar el producto a la lista `productos` de esta categoría; 2) asignar `producto.setCategoria(this)`, que es lo que realmente escribe la FK. `removeProducto(producto)`: 1) quitarlo de la lista; 2) asignarle otra categoría (o null si la FK lo permitiera; en mi caso no lo permite, porque es NOT NULL).

Pregunta 35 · CRÍTICA

¿Por qué una relación bidireccional puede generar recursión infinita al serializar JSON y cómo la evitas?

Qué responder

- Si se serializa la entidad: `Categoria` → `productos` → cada `Producto` → `categoria` → `productos`... Jackson recorre el ciclo sin fin (`StackOverflowError`). Además, con LAZY fallaría al tocar proxies fuera de la sesión.

- **Cómo lo evito: nunca expongo entidades JPA.** Los controllers devuelven **records DTO**: `ProductoResponse` lleva `categoriaId` (un número), no el objeto; `CategoriaResponse` no tiene lista de productos. No hay ciclo posible.
- Conozco `@JsonIgnore`, `@JsonManagedReference` y `@JsonBackReference`, pero no los necesito: acoplan el contrato de la API a la entidad de persistencia.

Muéstralo en el código

- `product/infrastructure/adapters/in/web/dto/ProductoResponse.java` (campo `Long categoriaId`).
- `category/infrastructure/adapters/in/web/dto/CategoriaResponse.java` (sin colecciones).

Repregunta de presión

¿Por qué 'poner `@JsonIgnore` hasta que deje de fallar' no sustituye un buen contrato DTO?

Porque la respuesta seguiría siendo la entidad: cualquier campo nuevo de la tabla se expondría sin querer, el contrato cambiaría cada vez que cambia el esquema y se mezclan anotaciones de JSON con las de persistencia. El DTO define **intencionalmente** qué ve el cliente.

Pregunta 36

¿Qué es el *persistence context* y qué significa *dirty checking*?

Qué responder

- El **persistence context** es la memoria de trabajo de Hibernate dentro de una transacción: guarda las entidades *managed* (una sola instancia por id) y una copia de su estado original.
- **Dirty checking**: al hacer flush o commit, Hibernate compara cada entidad managed con su estado original y genera un `UPDATE` sólo para las que cambiaron, **sin necesidad de llamar a `save()`**.
- Lo uso en el PUT: `ProductoPersistenceAdapter.actualizar` hace `findById` (managed) → `entity.actualizar(...)` → `repository.flush()` → `UPDATE`. No hay `save()`.

Muéstralo en el código

- `ProductoPersistenceAdapter.java:42-51` y `CategoriaPersistenceAdapter.java:32-39`.

Demuéstralo

Ejecuta el PUT de `requests.http` y muestra en la consola el `update stockflow.producto set ...` sin ningún `save()` en el código.

Repregunta de presión

Cargas una entidad, cambias un campo y no llamas `save()`: ¿puede actualizarse la base? ¿Bajo qué condiciones?

Sí, si la entidad está **managed**: se cargó **dentro de una transacción activa** y se modifica antes de que termine. Al hacer flush o commit se genera el `UPDATE`. Si la transacción es `readOnly` o la entidad ya está detached (fuera de la transacción), el cambio no llega a la base.

Pregunta 37

Diferencia los estados `transient`, `managed`, `detached` y `removed` de una entidad JPA.

Qué responder

- **Transient:** creada con `new`, sin id ni contexto. En mi código: `ProductoPersistenceMapper.toJpa` crea un `new ProductoJpaEntity(...)` con id null.
- **Managed:** dentro del persistence context; sus cambios se sincronizan. Pasa a serlo tras `save()/persist` en el POST o tras `findById` en el PUT.
- **Detached:** fue managed pero la transacción terminó. Con `open-in-view: false`, las entidades quedan detached apenas termina el método `@Transactional` del servicio; por eso los adaptadores las convierten a dominio antes.
- **Removed:** marcada para borrar con `remove` (mi `deleteById`); el `DELETE` se ejecuta al hacer flush.
- Operaciones: `persist` (`transient` → `managed`), `merge` (`detached` → copia `managed`), `remove` (`managed` → `removed`).

Muéstralo en el código

- `Transient` → `managed`: `ProductoPersistenceAdapter.guardar (:34-38)`. `Managed` y modificada: `actualizar (:42-51)`. `Removed`: `eliminar (:56-60)`.

Repregunta de presión

¿Qué pasa si modificas una entidad `detached` fuera de transacción?

Nada en la base: Hibernate ya no la está siguiendo. El cambio sólo existe en ese objeto hasta que se haga un `merge` dentro de una nueva transacción, que copia su estado a una instancia `managed`. Si además se accede a una relación `LAZY` no cargada, se lanza `LazyInitializationException`.

Pregunta 38

¿Qué es el problema `N+1` y cómo podrías detectarlo y resolverlo?

Qué responder

- Una consulta trae `N` entidades y luego, al acceder a una relación `LAZY` de cada una, se ejecuta **una consulta más por cada entidad**: `1 + N`.
- **Detección:** con los logs SQL (`org.hibernate.SQL: debug` en `application.yml:33`): si listar 100 elementos imprime 101 `SELECT`, hay `N+1`.
- **Solución según el caso:** `JOIN FETCH` en una consulta JPQL, `@EntityGraph`, **proyecciones o consultas DTO** (como mis `@Query` nativas del kardex, que traen todo en un solo `SELECT` con `JOIN`), paginación o batch fetching. **No** 'poner `EAGER`'.
- En StockFlow: `GET /api/productos/categoria/{id}` ejecuta **un** `SELECT`, porque el mapper sólo lee el id del proxy de la categoría. Si `GET /api/categorias` recorriera `categoria.getProductos()` en el mapper, sería `1 + N`; por eso los DTO no incluyen la colección.

Muéstralo en el código

- `ProductoPersistenceMapper.java:30` (`getCategoria().getId()`, sin `SELECT`).
- Consulta sin `N+1`: `SpringDataStockRepository.java:49-68` (una consulta con `JOIN` de tres tablas).

Repregunta de presión

Si listar 100 padres dispara 101 consultas, ¿qué te indica?

Que hay N+1: una consulta para los padres y una por cada padre para cargar su relación LAZY al mapearla. Hay que traer la relación en la misma consulta (fetch join, entity graph o DTO) o no recorrerla en el listado.

Pregunta 39

Explica spring.jpa.hibernate.ddl-auto y por qué 'update' no es una estrategia profesional de migración.

Qué responder

- Valores: `create` (borra y crea al arrancar), `create-drop` (además borra al cerrar), `update` (intenta alterar el esquema según las entidades), `validate` (sólo verifica que coincidan) y `none`.
- Uso `validate`: el esquema lo crean las **migraciones versionadas de Flyway** (V1 esquema, V2 semilla, V3 protección del histórico) y Hibernate sólo comprueba que Java coincida (ADR-002).
- `update` no es profesional: no borra ni renombra bien, no crea CHECK, triggers ni vistas, no deja historia ni se puede revisar en un PR, y cada máquina puede terminar con un esquema distinto.

Muéstralo en el código

- `application.yml:16` (`ddl-auto: validate`) y `:9` (`flyway`).
- `src/main/resources/db/migration/V1_..., V2_..., V3_...`; tabla `stockflow.flyway_schema_history` en DataGrip.

Repregunta de presión

Si dos desarrolladores cambian entidades distinto y ambos usan update, ¿cómo garantizas un esquema reproducible?

No se puede: cada base queda según el orden en que cada uno arrancó la aplicación. Con Flyway, el esquema es la secuencia de archivos **V1, V2, V3** en el repositorio; cualquiera los aplica en el mismo orden. Una migración aplicada no se edita (Flyway detecta el checksum distinto); se agrega una nueva.

Pregunta 40

Desde repository.save() hasta PostgreSQL: explica cómo JPA/Hibernate termina ejecutando SQL.

Qué responder

- `SpringDataProductoRepository` es una interfaz; Spring Data crea un **proxy** (`SimpleJpaRepository`). `save()` decide: si la entidad es nueva (id null), llama a `EntityManager.persist`; si no, a `merge`.
- El `EntityManager` es **Hibernate**: registra la entidad en el persistence context y genera el SQL (`INSERT` con `IDENTITY` de inmediato; los `UPDATE/DELETE` al hacer flush).
- Hibernate envía la sentencia por **JDBC** (pool HikariCP + driver de PostgreSQL); **PostgreSQL** ejecuta, aplica constraints y triggers y devuelve el resultado. El commit lo hace el proxy de `@Transactional` al terminar el servicio.
- `JpaRepository` **no es la base de datos**: es una abstracción sobre `EntityManager`.

Muéstralo en el código

- `ProductoPersistenceAdapter.java:38 (repository.save(entity)); application.yml:20,33 (format_sql, org.hibernate.SQL: debug).`

Demuéstralo

Haz un POST, muestra el `insert into stockflow.producto ...` en la consola y luego `SELECT * FROM stockflow.producto ORDER BY producto_id DESC` en DataGrip (`03_verificacion_y_pruebas.sql`, sección 5).

Repregunta de presión

¿Cómo demostrarías que un POST produjo realmente un INSERT y no sólo un objeto en memoria?

Con tres evidencias: 1) el `insert into ...` en el log SQL; 2) la respuesta trae un `id` generado por PostgreSQL; 3) la fila visible en DataGrip desde otra conexión, después del commit. Además, `PersistenciaPostgresTest` prueba la persistencia contra un PostgreSQL real.

Bloque D — DTO, record, servicios y repositorios

Pregunta 41

¿Por qué no conviene exponer tus @Entity directamente como contrato de la API?

Qué responder

- Acopla el contrato externo al esquema de la base: renombrar una columna rompería a los clientes web y móvil.
- Puede **filtrar campos internos**, generar **ciclos** de serialización (categoría ↔ productos), provocar consultas LAZY inesperadas o `LazyInitializationException`, y traer datos de más (*overfetching*).
- Con DTO decido **intencionalmente** qué entra y qué sale, y la API evoluciona sin depender de la persistencia.

Muéstralo en el código

- `ProductoJpaEntity` tiene `createdAt`, `updatedAt` y el objeto `categoria`; `ProductoResponse` no los expone y lleva `categoriaId`.

Repregunta de presión

Dame un campo de tu entidad que no debería viajar en la respuesta y explica por qué.

`updatedAt` (y `createdAt`) de `ProductoJpaEntity`: son auditoría técnica que mantiene Hibernate con `@UpdateTimestamp`, no información que el cliente necesite. Otro: la colección `productos` de `CategoriaJpaEntity`, que generaría un ciclo y consultas extra.

Pregunta 42

Diferencia Request DTO y Response DTO. ¿Por qué no siempre deben tener los mismos campos?

Qué responder

- **Request** = lo que el cliente **puede enviar**; **Response** = lo que el servidor **decide exponer**.
- `CrearProductoRequest` **no** tiene `id` (lo genera PostgreSQL) ni `activo` (nace activo). `ProductoResponse` **sí** tiene `id` y `activo`.
- `ActualizarProductoRequest` (PUT) tampoco tiene `id` (va en la ruta), pero **sí** `activo` (desactivar es una edición válida), y vuelve obligatorios `unidadMedida` y `stockMinimoDefault` porque es un reemplazo completo.
- Cada request tiene sus **validaciones de entrada** (`@NotBlank`, `@Pattern`, `@Digits`); el response no valida nada.

Muéstralo en el código

- `product/infrastructure/adapter/in/web/dto/`: `CrearProductoRequest`, `ActualizarProductoRequest`, `ProductoResponse`.

Repregunta de presión

En un PUT, ¿permitirías que el cliente cambie el id o una fecha de auditoría?

No. El **id** es la identidad: viene de la ruta y el servicio lo impone (`new Producto(id, ...)` en `ProductoService.actualizar`), ignorando cualquier otro. Las **fechas de auditoría** las pone el sistema (`@CreationTimestamp`, `@UpdateTimestamp`, y `created_at` es `updatable = false`); si el cliente pudiera cambiarlas, la auditoría dejaría de ser confiable.

Pregunta 43 · CRÍTICA

¿Qué es un record de Java y por qué es útil para DTOs?

Qué responder

- Un **record** declara un **portador de datos** en una línea: Java genera los campos **private final**, el constructor canónico, los accesorios (**codigo()**, sin **get()**), **equals**, **hashCode** y **toString**. Es **final** y no tiene setters.
- Encaja con un DTO: es un mensaje **inmutable por intención**; Jackson lo crea con el constructor canónico y las anotaciones de validación se ponen en sus componentes.
- La inmutabilidad **no es profunda**: si un componente es mutable (por ejemplo una **List**), su contenido sí se puede cambiar.
- En StockFlow **todos** los DTO son records (**CrearProductoRequest**, **ProductoResponse**, **ApiErrorResponse**, **PaginaResponse<T>...**) y también los modelos de lectura del dominio de inventario (**Existencia**, **MovimientoKardex**, **Pagina<T>**).

Muéstralo en el código

- **CrearProductoRequest.java:14** (**public record ...** con anotaciones en los componentes).
- **shared/web/ApiErrorResponse.java:7**.

Repregunta de presión

¿Usarías record como @Entity JPA? ¿Por qué un DTO record sí encaja?

No. JPA necesita un **constructor sin argumentos**, campos que pueda **modificar** (para cargar datos y para el dirty checking) y una clase **no final**, porque Hibernate crea **proxies** heredando de ella (LAZY). Un record es final, inmutable y sólo tiene constructor canónico. Un DTO, en cambio, se crea una vez y no cambia: encaja perfecto.

Pregunta 44

¿Dónde y cómo conviertes Entity ↔ DTO? Defiende tu estrategia de mapeo.

Qué responder

- Mapeo **manual**, con clases estáticas **final**, **una por frontera**: **ProductoWebMapper** (DTO ↔ dominio, en el adaptador web) y **ProductoPersistenceMapper** (dominio ↔ entidad JPA, en el adaptador de persistencia).
- En ningún punto se convierte entidad → DTO directamente: siempre pasa por el **dominio**. El controller sólo llama **toDomain/toResponse**, sin lógica.
- No uso MapStruct: con pocos campos, el mapeo manual es explícito y fácil de defender; el compilador avisa si cambia un constructor.
- Evita devolver relaciones enteras: el mapper de persistencia convierte el objeto **categoria** en **categoriaId**.

Muéstralo en el código

- **ProductoWebMapper.java:13, 26, 40**; **ProductoPersistenceMapper.java:12, 27**.

Repregunta de presión

Si agregas un campo nuevo al Response DTO, ¿qué capas revisas?

1) El record `ProductoResponse`; 2) `ProductoWebMapper.toResponse`; 3) si el dato es nuevo, el dominio `Producto`, `ProductoPersistenceMapper`, `ProductoJpaEntity` y una **migración nueva V4__...** (no se edita V1); 4) las pruebas y `requests.http`; 5) el README (contrato) y, después, los tipos del frontend.

Pregunta 45 · CRÍTICA

¿Qué responsabilidades pertenecen al Service/caso de uso y cuáles NO?

Qué responder

- **Sí:** orquestar el caso de uso, aplicar **reglas de negocio** de aplicación, definir el **límite transaccional** y coordinar puertos. `ProductoService.registrar`: la categoría debe existir (404) y estar activa (422), el código debe ser único (409, RN-06), y luego guarda; todo en una transacción.
- **No:** detalles HTTP (status, headers, `ResponseEntity`, DTO web), SQL o JPA concretos, ni formato JSON. Tampoco reglas de **forma** (eso lo hace `@Valid` en el DTO) ni invariantes del objeto (las aplica el constructor de `Producto`).
- Evito los dos extremos: ni *fat controller* (el controller sólo traduce) ni un servicio vacío que sólo reenvía llamadas.

Muéstralo en el código

- `ProductoService.java:39-48` (registrar) y `:53-68` (actualizar, con la regla de categoría inactiva sólo al cambiar de categoría).

Repregunta de presión

Muéstrame una regla de negocio real y explica por qué no debe vivir en el Controller.

'Una categoría inactiva no admite productos nuevos' (`ProductoService.java:43` y `:63`). No va en el controller porque cualquier otro adaptador (la app móvil, un job, un mensaje) debe cumplirla igual; si estuviera en el controller, otra entrada podría saltársela. Además así se prueba sin HTTP: `ProductoServiceTest.moverProductoACategoriaInactivaDa422`.

Pregunta 46

Diferencia un puerto de repositorio del dominio/aplicación y un `JpaRepository` de Spring Data.

Qué responder

- **Puerto** (`ProductoRepositoryPort`): interfaz del dominio, con lo que el núcleo **necesita**, en su idioma y con tipos de dominio (`Producto`, no `ProductoJpaEntity`).
- `JpaRepository` (`SpringDataProductoRepository`): interfaz de Spring Data para la que Spring genera un **proxy** con CRUD, consultas derivadas y traducción de excepciones a `DataAccessException`.
- El **adaptador** (`ProductoPersistenceAdapter`, `@Component`) implementa el puerto usando el `JpaRepository`. No uso `@Repository` en mis clases: los repositorios Spring Data ya son componentes y el proxy traduce las excepciones.

Muéstralo en el código

- `ProductoRepositoryPort.java:8` vs. `SpringDataProductoRepository.java:8`; adaptador: `ProductoPersistenceAdapter.java`.

Repregunta de presión**Si quieres probar el servicio sin PostgreSQL, ¿qué ventaja da el puerto propio?**

Puedo pasarle una implementación **en memoria**: `ProductoServiceTest` usa `FakeProductoRepository` implements `ProductoRepositoryPort` y ejecuta 15 pruebas en milisegundos, sin Spring ni base de datos. Implementar `JpaRepository` a mano para un test sería enorme.

Pregunta 47

Explica repository.findById(), Optional, consultas derivadas y @Query.

Qué responder

- `findById` ejecuta un SELECT por PK y devuelve `Optional<T>`. `Optional` hace explícita la ausencia: hay que tratarla (`orElseThrow`, `map`, `isEmpty`).
- **Consultas derivadas**: Spring genera el SQL a partir del **nombre del método**: `existsByCodigoAndIdNot`, `findByNombreContainingIgnoreCaseOrderByCodigoAsc`, `findByCategoria_IdOrderByCodigoAsc` (navega la relación hasta la FK).
- `@Query`: SQL escrito a mano cuando un nombre de método no alcanza. Uso `@Query(nativeQuery = true)` en `SpringDataStockRepository`: una consulta con JOIN de `stock`, `producto` y `ubicacion`, y el **kardex paginado** sobre la vista `vw_kardex`, con `countQuery` y `Pageable`.
- Una consulta de repositorio **no es** una regla de negocio: `existsByCodigo` sólo responde un dato; la regla 'el código es único → 409' está en el servicio.

Muéstralo en el código

- `SpringDataProductoRepository.java:13, 16`; `SpringDataCategoriaRepository.java:12-21`.
- `SpringDataStockRepository.java:49` y `:73-90` (`@Query` nativo, `countQuery`, `Page/Pageable`).
- `getReferenceById`: `ProductoPersistenceAdapter.java:36`.

Repregunta de presión**¿Qué diferencia hay entre findById(), getReferenceById() y una consulta custom?**

`findById`: intención 'quiero los datos'; ejecuta el SELECT y devuelve `Optional`. `getReferenceById`: intención 'sólo necesito apuntar a esta fila'; **no** hace SELECT, devuelve un proxy con el id (lo uso para llenar la FK al guardar un producto); si el id no existe, fallaría al usarlo o lo rechazaría la FK. **Consulta custom** (`@Query`): cuando necesito un resultado específico (JOIN, proyección, paginación) que no es 'una entidad por id'.

Bloque E — Validación, pruebas y evolución a microservicios

Pregunta 48

Separa tres niveles de validación: DTO/Bean Validation, reglas de negocio y constraints de PostgreSQL.

Qué responder

- **1. DTO (Bean Validation)** → forma y valores básicos de la entrada: `@NotBlank codigo`, `@Size(max = 40)`, `@Pattern`, `@PositiveOrZero stockMinimoDefault` → **400**.
- **2. Negocio (servicio y dominio)** → reglas que dependen del contexto o de otros datos: la categoría existe (404) y está activa (422), el código no está repetido (409), un solo almacén central (409). El constructor de `Producto` protege además sus invariantes (`DatoInvalidoException` → 422).
- **3. PostgreSQL** → integridad final: `uq_producto_codigo`, `fk_producto_categoria`, `NOT NULL`, `ck_producto_stock_minimo`, `ck_producto_unidad_medida`, triggers de V3. Se traduce a 409 (`DataIntegrityViolationException` en `GlobalExceptionHandler`).
- La duplicación es **intencional**: cada capa protege contra algo distinto. El DTO da un error claro por campo, el servicio una regla de negocio legible, y la base protege contra cualquier cliente y contra la concurrencia.

Muéstralo en el código

- `CrearProductoRequest.java` · `ProductoService.java:40-46` · `V1_creacion_completa_stockflow.sql:103-108` · `GlobalExceptionHandler.java:75-79`.

Repregunta de presión

Dos peticiones simultáneas pasan la validación 'no existe' y luego guardan el mismo valor único. ¿Qué capa resuelve la carrera?

PostgreSQL, con `uq_producto_codigo`. Las dos ven 'no existe' en `existePorCodigo`, pero sólo un INSERT puede confirmarse; el segundo viola el UNIQUE → `DataIntegrityViolationException` → mi handler responde **409**. La validación del servicio da el mensaje claro en el caso normal; la constraint garantiza la regla siempre.

Pregunta 49

¿Cómo probarías Controller, Service y Repository sin mezclar responsabilidades?

Qué responder

- **Dominio** (unitaria pura): `ProductoTest`, que prueba invariantes de `Producto` (RN-07, escala del precio, código obligatorio).
- **Servicio** (unitaria con *fakes*): `ProductoServiceTest`, `CategoriaServiceTest` y `UbicacionServiceTest` usan implementaciones en memoria de los puertos OUT. Prueban reglas: 404, 409, 422, idempotencia del PUT.
- **Controller** (prueba web): `ProductoControllerTest` con `@WebMvcTest` (sólo Spring MVC) y `@MockitoBean` para los casos de uso. Prueba el contrato HTTP: 201 + `Location`, 400 con `fieldErrors`, 404, 409, 422, 204 y 500 sin filtrar detalles.
- **Repository** (integración): `PersistenciaPostgresTest` con `@DataJpaTest` + **Testcontainers**, un PostgreSQL 16 real en Docker. Prueba lo que un mock no puede: Flyway V1-V3, `ddl-auto: validate`, `mappedBy/LAZY`, SQL nativo, paginación, FK → 409, triggers e índices.

- Unitarias = una pieza aislada, rápida; integración = varias piezas reales juntas. Mockear todo **oculta** errores de JPA/SQL: la prueba de integración encontró un error real (la fecha del kardex llegaba como `Instant`) que ningún mock habría detectado.

Muéstralo en el código

- `src/test/java/com/stockflow/: product/domain/model/ProductoTest, product/application/service/ProductoServiceTest, product/infrastructure/adapters/in/web/ProductoControllerTest, integration/PersistenciaPostgresTest.`
- Ejecutar: `mvn test` (57 pruebas). Sin Docker, las 10 de integración se omiten.

Repregunta de presión

Dame una prueba que detectaría un error en `mappedBy` que una prueba unitaria del service nunca detectaría.

`PersistenciaPostgresTest`: al arrancar construye el `EntityManagerFactory`. Con un `mappedBy` mal escrito, Hibernate lanza `AnnotationException` ahí y **toda la clase falla** (lo comprobé). Además, `LadoInversoMappedByLeeLosProductosATravesDeLaFkYEsLazy` verifica que la colección traiga los productos correctos. `ProductoServiceTest` nunca lo vería: no usa JPA.

Pregunta 50 · CRÍTICA

Defiende por qué un monolito hexagonal modular es una base sólida para empezar y cómo facilita un salto futuro a microservicios.

Qué responder

- **Por qué empezar así:** un solo despliegue, una base, transacciones locales, depuración simple, sin red entre servicios. Pero con **límites claros**: módulos por negocio, puertos y adaptadores, y un módulo usa a otro sólo por su **puerto IN** (`ProductoService` → `ConsultarCategoriaUseCase`; `InventarioConsultaService` → `ConsultarProductoUseCase` y `ConsultarUbicacionUseCase`).
- **Cómo facilita el salto:** si un módulo ya se comunica por un puerto IN, extraerlo consiste en reemplazar la implementación local de ese puerto por un **cliente HTTP** y darle al nuevo servicio su propia base. El resto del código no se entera.
- **No es automático.** Hacen falta fronteras de negocio reales, contratos (API), **independencia de datos**, observabilidad, despliegue propio y manejo de **consistencia distribuida** (lo que hoy resuelve una transacción local pasaría a ser eventual).
- **Ejemplo, extraer `location`:** quedaría su API (`/api/ubicaciones`) como contrato; su tabla `ubicacion` pasaría a su propia base; en `inventory`, `ConsultarUbicacionUseCase` se implementaría como cliente HTTP. El reto es que `stock` y `movimiento_inventario` tienen FK a `ubicacion`: esas FK desaparecerían y la integridad pasaría a validarse por API, con consistencia eventual.

Muéstralo en el código

- Dependencia entre módulos por puerto IN: `ProductoService.java:29`; `InventarioConsultaService.java:28-29`.
- `docs/03-decisions/ADR-001-monolito-modular-hexagonal.md` (sección Consecuencias).

Repregunta de presión

Si dos módulos comparten tablas, llaman a repositorios del otro y tienen ciclos, ¿están 'listos para microservicios' aunque estén en paquetes separados?

No, y en mi proyecto hay un caso que debo reconocer: en infraestructura, `product` y `category` tienen un **ciclo a nivel de entidades JPA** (`@ManyToOne` y `@OneToMany(mappedBy)`) y `ProductoPersistenceAdapter` usa `SpringDataCategoriaRepository`. Está confinado a infraestructura: dominio y servicios sólo usan el puerto `ConsultarCategoriaUseCase`. Para extraer `product`, lo primero sería reemplazar `@ManyToOne CategoriaJpaEntity` por un `Long categoriaId`, como ya hace el módulo `inventory` en `StockJpaEntity`. Los paquetes separados no bastan: lo que importa son las **dependencias reales**.

Los 5 desafíos prácticos

No reemplazan las 50 preguntas: comprueban que dominas el código y no sólo el discurso.

Desafío 1 — Trazado en vivo

Elegir un POST real, abrir Controller, DTO, Service, mapper, Repository y Entity, narrar el flujo sin ejecutar, y luego comprobar en logs y en la base.

Cómo resolverlo en StockFlow

- Usa `POST /api/productos` (el de `requests.http` que guarda `{{productoId}}`).
- Abre en este orden: `ProductoController.crear` → `CrearProductoRequest` → `ProductoWebMapper.toDomain` → `Producto` (constructor) → `ProductoService.registrar` → `ProductoPersistenceAdapter.guardar` → `ProductoPersistenceMapper.toJpa` → `ProductoJpaEntity`.
- **Predice antes de ejecutar:** status 201, header `Location: /api/productos/{id}`, un SELECT de la categoría (`buscarPorId`), un SELECT del `exists` por código y un `insert into stockflow.producto`. No habrá SELECT del proxy de categoría (`getReferenceById`).
- Ejecuta y compara con la consola (`org.hibernate.SQL: debug`). Luego muestra la fila en DataGrip.

Desafío 2 — Romper una anotación

Predecir qué ocurriría al eliminar `mappedBy`, `@Valid`, `@Service` o `@JoinColumn`.

Cómo resolverlo en StockFlow

- `mappedBy` (`CategoriaJpaEntity`): Hibernate trata el `@OneToMany` como un segundo lado propietario y busca una tabla intermedia → no arranca: `Schema-validation: missing table [categoria_productos]`. Con `mappedBy = "categoria_id": AnnotationException ... 'mappedBy' a property named 'categoria_id' which does not exist`. **Comprobados en el proyecto.**
- `@JoinColumn` (`ProductoJpaEntity`): Hibernate infiere `categoria_categoria_id` → `Schema-validation: missing column [categoria_categoria_id] in table [stockflow.producto]`. **Comprobado.**
- `@Valid` (`ProductoController.crear`): no se valida el DTO; un código vacío llega al dominio → `DatoInvalidoException` → **422** en vez de 400 con `fieldErrors`. Un stock mínimo negativo → `StockMinimoInvalidoException` → 422.
- `@Service` (`ProductoService`): no hay bean que implemente `RegistrarProductoUseCase` → el arranque falla con `NoSuchBeanDefinitionException` al crear `ProductoController`.

Desafío 3 — Error intencional

Enviar un payload inválido, un id inexistente y un dato duplicado; anticipar status, capa y respuesta.

Cómo resolverlo en StockFlow

- **Payload inválido** (`codigo: ""`): lo detecta Bean Validation (adaptador web) → **400**, `message: "Datos de entrada inválidos"`, `fieldErrors.codigo`.
- **Id inexistente** (`GET /api/productos/9999`): lo detecta el controller con `orElseThrow` → `ProductoNoEncontradoException` → **404**, `message: "No existe el producto con id: 9999"`.
- **Padre inexistente** (`categoriaId: 9999` en un POST): lo detecta el servicio → **404**.
- **Duplicado** (`codigo: "PRD-OFI-001"`): lo detecta el servicio con `existePorCodigo` → **409**, `message: "RN-06: ya existe un producto con el código PRD-OFI-001"`.

- La estructura siempre es `ApiResponseResponse: timestamp, status, error, message, path, fieldErrors`. Todos los casos están en `requests.http`.

Desafío 4 — SQL y cardinalidad

Crear un padre con hijos, mostrar PK/FK; luego borrar o desvincular un hijo y explicar el SQL según `cascade/orphanRemoval`.

Cómo resolverlo en StockFlow

- Crea una categoría (POST, guarda `{{categoriaId}}`) y un producto en ella. En DataGrip: `SELECT p.producto_id, p.categoria_id, c.categoria_id FROM stockflow.producto p JOIN stockflow.categoria c ON c.categoria_id = p.categoria_id`; y señala la FK.
- **Desvincular un hijo** = cambiarlo de categoría con `PUT /api/productos/{id}` y otro `categoriaId`. SQL esperado: `UPDATE stockflow.producto SET categoria_id = ?, ... WHERE producto_id = ?` (dirty checking). No se puede dejar en NULL: la FK es NOT NULL.
- **Borrar un hijo** = `DELETE /api/productos/{id}` → `DELETE FROM stockflow.producto WHERE producto_id = ?` → 204 (o 409 si tiene stock o movimientos).
- **Borrar el padre con hijos** → 409. Como **no hay** `cascade ni orphanRemoval`, Hibernate no genera ningún DELETE de hijos: sólo intenta el DELETE de la categoría y PostgreSQL lo rechaza por `fk_producto_categoria`. Con `CascadeType.REMOVE` habría un DELETE por cada producto antes; con `orphanRemoval`, quitar un producto de la lista generaría su DELETE.

Desafío 5 — Extracción futura

Elegir una funcionalidad y describir cómo convertirla en microservicio: contrato, datos, dependencias, transacción y comunicación.

Cómo resolverlo en StockFlow

- Elige `location` (Ubicaciones), el módulo más limpio: no importa entidades JPA de otros módulos.
- **Contrato:** su API actual, `POST/GET /api/ubicaciones`, documentada en Swagger. **Datos:** la tabla `ubicacion` pasa a su propia base. **Dependencias:** `inventory` usa `ConsultarUbicacionUseCase`; se implementa como cliente HTTP (con timeout y manejo de errores) y el resto del código no cambia.
- **Transacción:** las FK de `stock` y `movimiento_inventario` hacia `ubicacion` desaparecen; validar que la ubicación exista pasa a ser una llamada remota y la consistencia es eventual. **Comunicación:** REST síncrono para consultas; eventos ('ubicación desactivada') si otros módulos deben reaccionar.
- Para extraer `product` primero habría que romper el ciclo JPA con `category` (ADR-001, ADR-003).

Señales de que todavía no estás preparado

- Puede crear el CRUD, pero no sabe explicar qué sucede entre el Controller y PostgreSQL. → Practica la pregunta 11 dibujando.
- Dice que Swagger "es el backend" o que Postman "guarda en la base". → Pregunta 14.
- Confunde Entity con DTO. → Preguntas 3, 41, 42.
- No sabe en qué tabla está la FK de su 1:N. → Pregunta 28: está en `producto.categoria_id`.
- No sabe quién es el lado propietario ni qué significa mappedBy. → Pregunta 30.
- Usa CascadeType.ALL, EAGER o @JsonIgnore como recetas. → Preguntas 31, 33, 35.
- No puede explicar por qué usa record. → Pregunta 43.
- No distingue validación, regla de negocio y constraint. → Pregunta 48.
- No sabe qué status devuelve cada error. → Pregunta 20 y la tabla de códigos HTTP.
- No puede dibujar POST/GET/PUT/DELETE sin apuntes. → Preguntas 15, 16, 18, 19.
- Su Controller accede directamente a JPA. → Pregunta 5: en StockFlow ningún controller importa un JpaRepository.
- Afirma que por tener paquetes separados ya está listo para microservicios. → Pregunta 50 (el ciclo entre product y category).

Checklist final (márcalo sin mirar la guía)

- Puedo explicar el flujo completo de POST, GET, PUT y DELETE usando mi propio código.
- Puedo explicar todas las anotaciones que aparecen en mis clases principales.
- Puedo dibujar categoria y producto con su PK/FK (fk_producto_categoria) sin mirar el código.
- Puedo explicar lado propietario, mappedBy ("categoria"), @JoinColumn ("categoria_id"), cascade, orphanRemoval y fetch.
- Puedo justificar por qué uso DTO record y por qué no expongo entidades.
- Puedo explicar qué hace ProductoService y qué hace ProductoPersistenceAdapter.
- Puedo explicar dónde empieza y termina la transacción y qué hace Hibernate dentro de ella (dirty checking).
- Puedo diferenciar 400, 404, 409, 422 y 500 con ejemplos de StockFlow.
- Puedo demostrar con el log SQL y con DataGrip que una operación llegó a PostgreSQL.
- Puedo explicar cómo el monolito modular facilita una separación futura sin decir que la hace automática.

Recuerda: el docente evalúa si **tú** entiendes el sistema. Si una respuesta de esta guía no la puedes explicar con tus palabras y mostrar en IntelliJ, todavía no está aprendida. Practica en voz alta, con el proyecto abierto, y pídele a un compañero que te haga las repreguntas.